

Week 3 考卷：光線與紀錄

作品可現場操作或用手機展示，完成後回答題末問題。可查資料或使用 AI。

作品：在電腦顯示光線讀值

用課堂 ESP32-S3 開發板與光敏模組，讓電腦持續顯示光線讀值。

特色：在 Arduino IDE 的 Serial Monitor (序列監控視窗) 觀察光線變化。開發板上的 ESP32 晶片把光敏訊號電壓轉成整數，這個數字叫原始讀值 **raw**；本題顯示範圍為 0 ~ 4095。

改線前拔 USB 與其他電源；先確認接線與供電相容，不將 5 V 接入 ESP32 訊號腳。不確定接法時保持斷電，不通電試錯。

預期結果：Serial Monitor 持續出現 0 ~ 4095 的整數，可讀到沒遮光與遮光時的數值；變化方向與數字以本組實測為準。

驗證方法：固定感測器、燈與接線的位置，觀察「沒遮光」與「用不透光紙遮住」時的數字，各記連續三筆 raw。

Q1 · 你的作品在兩種光線下各讀到多少？

當時的情況	第 1 筆 raw	第 2 筆 raw	第 3 筆 raw
沒遮光	420	425	418
遮光	1000	1010	1005

Q1 參考填法：上列是示例，不是本組實測。正確作答須與本組畫面一致；沒有全班共用的固定數字。

Q2 · raw=800 可以寫成 0.8 V 嗎？

請說明原始讀值 800 代表什麼，以及你的判斷理由。

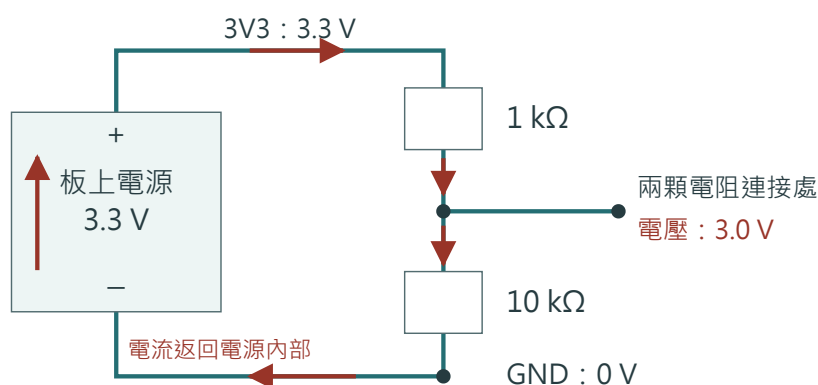
Q2 參考答案：不能。800 是 ESP32 的 ADC 將接腳電壓轉成的整數，不是以毫伏特表示的 800。換成伏特需使用對應 ADC 設定的換算與校正資料，不能直接除以 1000。

Q3 · 電表會量到多少電壓？

紙上題：在圖上作答，不用重新接線。

開發板的 3V3 電源腳提供 3.3 V，GND 接地腳作為 0 V。兩腳之間串接兩顆電阻：靠近 3V3 腳的是 1 k Ω ，靠近 GND 腳的是 10 k Ω ；1 k Ω = 1000 Ω 。

要量的位置：兩顆電阻連接處與 GND 之間。圓點是導線接點，兩個小長方形是電阻；採標示阻值計算，忽略電表的影響。



1. 畫電流：在導線與電源內加箭頭，標出傳統電流繞完整回路的方向。
2. 算電壓：兩顆電阻連接處比 GND 高多少 V？寫出計算式，填入圖中空格。

Q3-2：電流 $I = 3.3 \div (1000 + 10000) = 0.0003$ A。
 下方電阻電壓差 $V = I \times R = 0.0003 \times 10000 = 3.0$ V。

3. 設定電表：用 A830L 萬用電表量上述兩點，讓電表顯示正值。表筆一端的插頭插入電表，另一端的金屬筆尖碰測量位置。

問題	你的答案
黑色表筆的插頭插入電表哪個孔？	電表 COM 插孔
紅色表筆的插頭插入電表哪個孔？	電表 V Ω mA 插孔
電表旋鈕選哪種量測、哪個範圍？	直流電壓，20 V 範圍
黑色筆尖碰圖中的哪裡？	GND 接地點
紅色筆尖碰圖中的哪裡？	兩顆電阻連接處

Q4 · 這樣接表筆，能量電壓嗎？

只在紙上回答，不把錯誤接法接到實物。

同學想用 **A830L 萬用電表**，量 ESP32 開發板的 3V3 電源腳與 GND 接地腳之間的電壓。目前開發板尚未接上電源。

先分清楚：插頭插電表，筆尖碰電路

這支電表下方有三個表筆插孔，旁邊分別印著 **10A**、**COM**、**VΩmA**。其中 10A 是電流量測插孔的標字，A 是電流單位「安培」；不是麵包板孔號，也不是本次量到的電流。

同學目前的接法

檢查哪個部分	同學的設定或打算碰的位置
黑色表筆的插頭	插入萬用電表上標示 COM 的插孔
紅色表筆的插頭	插入萬用電表上標示 10A 的插孔
萬用電表的旋鈕	選擇直流電壓的 20 V 量測範圍
黑色表筆的金屬筆尖	準備碰開發板的 GND 接地腳
紅色表筆的金屬筆尖	準備碰開發板的 3V3 電源腳

同學認為旋鈕已選「量電壓」，就可以接上開發板的 USB 電源開始量測。

為什麼仍應阻止他供電？ 指出接法中的問題，畫出或寫出供電後可能形成的電流路徑，解釋危險的原因。

Q4 參考答案：紅色表筆插頭誤插電表的 **10A 電流插孔**。旋鈕選量電壓，仍不能代替插孔核對。

可能形成：開發板 3V3 → 紅色筆尖與導線 → 電表 10A 插孔 → 電表內部低電阻路徑 → COM 插孔 → 黑色導線與筆尖 → GND。

這條路徑可能造成短路。保持開發板斷電，紅色插頭改接電表 **VΩmA** 插孔，再核對檔位和測點；不能通電試錯。**10A** 標字不代表實際電流必定為 10 安培。

從光線到紀錄，中間經過什麼？

ESP32-S3 開發板連著光敏模組和一顆按鈕。模組的電源端、接地端分別接開發板的 3V3、GND 腳；模組上標示 S 的訊號腳接開發板的 GPIO4 腳。按鈕兩端接開發板的 GPIO5、GND 腳，GPIO5 使用晶片內的上拉電阻讀取按鈕。

每次新的按下，程式讀取一次 GPIO4，在 Serial Monitor (Arduino 的文字視窗) 顯示第幾筆、raw，以及程式自動記下的「讀到這筆時，距離開機多久」。按住不放不重複記錄，放開再按才增加下一筆。

USB 線插在開發板背面標示 COM 的 USB 接頭，另一端接電腦。此處 COM 是開發板接頭的標字，不是電表插孔；開發板上的 CH343 晶片把 ESP32 經 UART 串列通訊送出的資料轉成 USB 資料。

Q5-1 · 畫出資訊流

依上述條件，從光線改變開始，畫到電腦出現一筆資料。另畫按鈕的資訊在哪裡加入。標出接腳、電壓轉成 raw 的位置及資料到電腦的路徑；每支箭頭註明傳遞什麼。

Q5-1 參考答案：

光線變化 → (感光元件阻值改變) → 光敏模組 S 腳的電壓改變 → (訊號電壓) → 開發板 GPIO4 → (晶片內 ADC 轉換) → raw 整數 → (程式加入序號與時間) → 一筆紀錄。

按鈕按下 / 放開 → (高低電壓) → GPIO5 → (程式辨識新的按下) → 觸發上方的 ADC 取樣與紀錄。

紀錄 → (Serial 文字 / UART 串列資料) → CH343 → (USB 資料) → 電腦 Serial Monitor。箭頭表示資訊，不是供電電流。

Q5-2 · 只斷開 S 訊號線

在紙上把「光敏模組 S 訊號腳到開發板 GPIO4 腳」的導線畫斷，其他供電、接線與程式不變。Serial Monitor 仍出現數字，能否當成光線資料？能否保證變成 0？請解釋原因。

Q5-2 參考答案：不能當成有效光線資料，也不能保證讀到 0。訊號線斷開後，GPIO4 不再接到感測器輸出；若沒有其他電路固定它的電壓，就會浮接，可能受周圍干擾而讀到不同數字。模組有供電，不表示訊號仍送到 GPIO4。

把光線讀值做成可以使用的作品

Q1 已讓電腦顯示光線數字。接下來加入按鈕或 OLED 小螢幕，讓使用者能決定何時記錄，或直接看到光線的變化。

先做 A：按一下，自動記錄三次

按鈕只按一次，作品就自動量三次光線，並把三筆結果顯示在電腦。這是同一段短時間內的三次量測。

再看三個延伸作品 B、C、D

- **B · 可以暫停的遮光計數器**：用按鈕啟動或暫停，計算感測器被遮住幾次。
- **C · 桌面光線觀測器**：不用按按鈕，在 OLED 看目前讀值，以及這次開機後的最小、最大值。
- **D · 按一下，留下光線快照**：OLED 持續顯示目前值；按一下，另外保留當時的值，方便比較前後變化。

各題是分開展示的作品，不必把所有功能塞進同一支程式。可以沿用已有的零件與程式，但每次展示都要符合該題的行為。

每題怎樣算完成？

先看「作品」與「預期結果」，完成後依「驗證方法」操作。現場或用手機展示畫面與反應，最後回答題末問題，說明自己的程式如何完成要求。

預期畫面中的光敏數字只是示例；作品顯示的是自己的實際量測，不要求做出相同數字。

A · 按一次，自動取三筆

作品：用光敏模組與一顆按鈕做光線取樣器。「取一筆」就是重新量一次當下光線，記下讀值。按一下後，程式自動取三筆並顯示在電腦，不必連按三下。

特色：辨識到新的按下就取第一筆；至少隔 0.2 秒取第二筆，再至少隔 0.2 秒取第三筆，之後停止。計時由程式完成，不靠人抓時間。

改線前拔除 USB 與其他電源，換程式前先拆外接線。確認接線、供電及腳位用途後再接回；不要把電源腳或設定為高電壓輸出的 GPIO 接腳 直接接 GND。

預期結果：按一下，電腦新增三行

以下是假設第一次按下的畫面。時間從開機起算，單位是毫秒；1000 毫秒 = 1 秒。光敏數值與時間都是示例。

第幾次開始	這次第幾筆	光敏 raw	讀取時間 (毫秒)
1	1	420	5000
1	2	425	5200
1	3	418	5400

同一次開始的三筆分別來自三次量測，不是把第一筆複製三次。光線沒變時，三筆數字也可能相同，不能只用數字不同來判斷是否重新量測。

三筆讀完前後，按鈕應該怎麼反應？

- 按住不放或提早放開，都只完成這次三筆，之後停止。
- 尚未讀完又按一下，顯示「仍在讀取，這次按下不接受」；原三筆照常完成，不在之後補做。
- 三筆完成後放開再按，開始第二次，新增第 1~3 筆；畫面能區分兩次。
- 按開發板的 RST 重新啟動按鈕會中止讀取；先放開再按，重新從第一次開始。

A · 怎樣確認作品完成？

驗證方法：先展示正式的 0.2 秒間隔，再暫時放慢，檢查讀取途中按鈕仍能回應。時間由程式顯示，不用手動計時或抄時間。

1. 正式間隔：一次三筆，兩次共六筆

先放開按鈕、按開發板的 RST 按鈕，再按住按鈕兩秒。畫面只新增三筆。放開後再按一次，才新增下一次的三筆；每次第二、三筆與前一筆的時間差都至少是 200 毫秒（0.2 秒）。

2. 暫時放慢：讓手動操作來得及

把作品暫改為兩筆相隔約一秒，筆數仍是三筆。以下每列分開測試；開始前先放開按鈕，再按開發板的 RST 按鈕。

操作	作品應有的結果
按一下並放開；第一筆出現後遮住感測器	仍完成三筆；後兩筆反映遮住後的光線
按下；第一筆出現後放開約半秒，再按住	顯示仍在讀取的提示；原三筆完成後停止，不補做
按下；第一筆出現後按開發板的 RST 按鈕	這次讀取中止；放開再按，重新從第一次開始

測完恢復 0.2 秒，再確認一次三筆。慢速測試用來看操作反應，不用它證明正式間隔正確；換程式依前頁安全提醒操作。

A · 你的程式如何完成三次取樣？

從自己的程式指出「重新量測光線」的敘述，說明第一筆與後兩筆各在什麼時候執行、何時停止。用關鍵程式片段加文字說明，如何避免複製第一筆。

A 參考答案：capture(now) 每次執行 analogRead(PIN_LIGHT) 重新取樣。空間時辨識新的按下，先讀第一筆；之後距前一筆至少 200 毫秒再讀，第三筆後停止。每次都重新讀感測器，不複製第一筆。這是參考程式的寫法；其他實作須說明自己程式中的對應敘述與時機。

B · 可以暫停的遮光計數器

作品：用光敏模組與一顆按鈕做遮光計數器。每次由未遮住變成遮住，計數一次；Serial Monitor 顯示啟動 / 暫停狀態與次數。

按鈕啟動後，遮住一次

```
mode=RUNNING  
count=1
```

暫停後再遮住，不增加

```
mode=PAUSED  
count=1
```

Serial Monitor 預期內容示意；不是實測截圖，文字位置與字型可自行設計。

使用者看到的資訊	意義
mode=RUNNING / mode=PAUSED	畫面上的 mode 欄顯示正在計數 / 暫停計數
count	畫面上的次數欄：這次開機後，計數期間完成的遮光次數
一次遮光	從未遮住變成遮住；持續遮住不算新的一次

特色：按一下按鈕切換啟動 / 暫停；按住不放只切換一次。暫停保留次數，但暫停期間的動作不補算。按開發板的 RST 按鈕後回到 PAUSED、count=0，不要求斷電保存。

判斷依據：接線與位置相同時，可沿用 Q1 的未遮光 / 遮光資料；兩種情況必須能區分。條件改變才重新量測，不直接套用別組的分界。

改線前拔除 USB；供電、訊號電壓與 GPIO 仍須符合已確認的器材規格。不要把電源腳或設定為高電壓輸出的 GPIO 接腳直接接 GND。

B · 怎樣確認作品完成？

驗證方法：從未遮住、按鈕放開的狀態按開發板的 RST 按鈕，依序做下表操作，展示狀態與次數。

遮住或移開至少一秒，避開分界附近的光線。除第 8 步外，按鈕按一下就放開。

順序與操作	畫面應顯示的狀態與遮光次數
1. 啟動後不操作	PAUSED / 0
2. 按一下按鈕	RUNNING / 0
3. 遮住並保持三秒	RUNNING / 1，不持續增加
4. 移開，再遮住	RUNNING / 2
5. 保持遮住，按一下暫停	PAUSED / 2
6. 暫停時移開、遮住兩次	PAUSED / 2
7. 遮住時恢復，再移開、遮住	RUNNING / 2 → RUNNING / 3
8. 仍在 RUNNING，按住按鈕兩秒	只切換成 PAUSED / 3，不反覆切換
9. 放開按鈕，再按開發板的 RST 按鈕	PAUSED / 0

模式或次數改變時，Monitor 留下紀錄。開機已遮住不加次數；按鈕按住開機，放開再按才接受。模式切換不算遮光，暫停中的動作不補算。

B · 你的程式如何判斷現在被遮住？

寫出自己的判斷規則，包含比較方向與分界數字，並引用本組實測作為依據；條件相同可用 Q1。可用程式或文字回答。

B 參考答案 (沿用 Q1 示例)：沒遮光 418 ~ 425、遮光 1000 ~ 1010，遮光時數字較大。分界可取 712；在 1 ~ 4094 內， $raw > 712$ 判遮光，否則判沒遮光。0、4095 先不判定。自己的答案須用本組量測決定分界與比較方向。

C · 桌面光線觀測器

作品：用光敏模組與 OLED 做光線觀測器。不用按按鈕，就能看到目前讀值，以及這次開機後量到的最大、最小值。

剛開始，第一筆 raw=480

```
RAW 480
MIN 480
MAX 480
```

之後量到 980，再回到 510

```
RAW 510
MIN 480
MAX 980
```

OLED 預期畫面示意；不是實測截圖，文字位置與字型可自行設計。

畫面資訊	需要呈現的內容
RAW	OLED 畫面上顯示最近一筆光敏原始讀值的欄位
MIN	OLED 畫面上顯示本次開機後最小讀值的欄位
MAX	OLED 畫面上顯示本次開機後最大讀值的欄位

特色：畫面至少每秒更新一次。MIN、MAX 包含最新一筆資料；光線回復時，兩者不跟著回到目前值。按開發板的 RST 按鈕 後重新開始統計，不要求斷電保存。

480、980、510 只是示例，假設中間沒有其他更大或更小的讀值。自己的作品顯示自己的量測結果，不要把示例數字寫死。

改線前斷電；確認 OLED 的供電與訊號電壓相容，不把模組接到 GPIO 當電源。不確定接法時保持斷電，不提高電壓試錯。

C · 怎樣確認作品完成？

驗證方法：改變光線，觀察目前值與最大、最小值；再按開發板的 RST 按鈕，確認重新開始記錄。以自己的讀值核對下表。

操作或條件	預期畫面與紀錄
剛啟動，尚無第一筆資料	螢幕可顯示 WAIT (等待資料) ；不能把未取得 的資料當成量測值
取得第一筆資料	RAW、MIN、MAX 都等於這一筆
維持相同光線	RAW 可小幅變動；範圍包含所有已取得的值
raw 高於目前 MAX	MAX 更新；MIN 保留
raw 低於目前 MIN	MIN 更新；MAX 保留
raw 回到已記錄範圍中間	RAW 改變；MIN、MAX 保留
按開發板的 RST 按鈕	舊範圍清除；以重啟後第一筆重新開始

Serial Monitor 至少每秒留下一筆相同資料組的 raw、min、max；現場或手機展示 OLED 與 Monitor 的同一次更新。

範圍包含所有取得的讀值，須符合 $\text{MIN} \leq \text{RAW} \leq \text{MAX}$ 。

C · 你的程式如何保留最小值與最大值？

寫出自己更新 MIN、MAX 的關鍵程式，說明第一筆與後續讀值各如何處理。

C 參考答案：每次先讀新的 raw。第一筆執行 `minimum = maximum = raw;`，用真實讀值建立範圍。

之後執行 `if (raw < minimum) minimum = raw;` 及 `if (raw > maximum) maximum = raw;`。落在範圍內時，最小值與最大值都不變。兩個變數持續保留到重新開機，不在每次迴圈歸零。

D · 按一下，留下光線快照

作品：用光敏模組、按鈕與 OLED 做可保存光線讀值的顯示器。畫面持續顯示目前讀值；按一下按鈕，另外保留按下時的讀值，供之後比較。

尚未按過按鈕

RAW 480
LAST ---
SAVED 0

保存 480 後，光線變為 980

RAW 980
LAST 480
SAVED 1

OLED 預期畫面示意；不是實測截圖，文字位置與字型可自行設計。

畫面資訊	需要呈現的內容
RAW	OLED 的目前讀值欄，至少每秒更新一次
LAST	OLED 的最近保存值欄；尚未保存時顯示 ---
SAVED	OLED 的保存次數欄，這次開機後從 0 開始

特色：每次新的按下保存一筆，LAST 更新、SAVED 加一；按住不放只保存一次。保存後改變光線，RAW 繼續變，LAST 保持不變，直到下一次按下。

每次保存都在 Serial Monitor 留下一筆序號與讀值。只保留最近一筆即可，不要求全部歷史、資料庫或網路。按開發板的 RST 按鈕或重新接上電源後，LAST 回到 ---、SAVED 回到 0。

OLED 規格須先確認；光敏、按鈕及 OLED 訊號不得誤用同一 GPIO。改線前斷電，不把 OLED 或其他負載接到 GPIO 當作供電。

D · 怎樣確認作品完成？

驗證方法：按鈕先放開，再按開發板的 RST 按鈕，依下表改變光線並操作按鈕。表中數值為示例，展示時使用自己的讀值。

順序與操作	OLED 預期結果
1. 啟動，raw 為 480	RAW 480 ; LAST --- ; SAVED 0
2. 按一下，此次取得 480	RAW 持續更新 ; LAST 480 ; SAVED 1
3. 不再按鈕，改變光線至 980	RAW 980 ; LAST 480 ; SAVED 1
4. 放開再按，此次取得 980	LAST 980 ; SAVED 2
5. 繼續按住三秒，光線改至 510	RAW 510 ; LAST 980 ; SAVED 2
6. 放開再按，此次取得 510	LAST 510 ; SAVED 3
7. 放開按鈕，再按開發板的 RST 按鈕	RAW 顯示新讀值 ; LAST --- ; SAVED 0

第 2、4、6 步各新增一筆保存紀錄；其餘步驟不新增保存事件。第 7 步清除本次進度，但電腦上已印出的舊文字不會因此消失。

OLED 的 LAST 必須與最新一筆保存紀錄相同，SAVED 必須與序號相同；LAST 與即時 RAW 不必相同。按鈕按住開機時，放開後再按才保存第一筆。

成品條件：OLED 文字清楚、不重疊或超出畫面；排版可自行設計。

D · 你的程式如何分開更新目前值與保存值？

指出自己用哪些變數保存 RAW 與 LAST，並說明各在什麼情況下更新。可用關鍵程式片段加文字回答。

D 參考答案：raw 保存目前讀值，持續重新取樣；savedRaw 保存最近一次按下時的讀值。只有辨識到新的按下時才執行 `savedRaw = raw;`，並將 `savedCount` 加一。

OLED 的 RAW 顯示 raw，LAST 顯示 savedRaw；未保存前顯示 ---。未再按下時只更新目前值，按住不放不重複保存。

逐題詳解：接線、程式與原理

前 13 頁對照 Main 的題目與作答位置；從本頁開始看完整做法。

紅字是參考答案。Q1 實測值與 B 的分界須以本組資料為準；A、C、D 的程式寫法不只一種，但必須符合題目行為。考卷中的成品畫面與預期結果仍是驗證依據，不代表作品已經實測完成。

題目	詳細說明
共通準備：開發板、電表與程式	p15 ~ 26
Q1：取得沒遮光、遮光的 raw	p27 ~ 35
Q2：raw 與電壓；延伸到 B 的分類	p36 ~ 41
Q3：分壓、電流與量測位置	p42 ~ 48
Q4：錯誤表筆接法與短路風險	p49；電表操作見 p19
Q5：按鈕觸發、資訊流與斷線	p50 ~ 59
A：一次按下，重新量三筆	p60 ~ 64
B：判斷遮光、計次與暫停	p65 ~ 70
C：OLED 與最小、最大值	p71 ~ 77
D：分開保存目前值與按下時的值	p78 ~ 80

從頭實作時，先完成共通準備，再依題目往下做；查特定題目時，可用上表直接找到說明。改線前先斷電，必要接線與上傳提醒保留在各操作步驟旁。

先準備量測，再完成每一題

Week 3：用電表看到電壓，再讓 ESP32 讀到變化。

用一張不透光的紙遮住光敏模組，觀察電表與電腦讀值如何改變，再讓程式印出「室內光 / 遮光」。



我們依序做	要看到的結果
共通準備：設定電表，量 3V3 和 GPIO5	電源電壓，以及程式切換的高、低電壓
Q1 ~ Q2：接光敏模組，遮住再移開	記錄自己的 raw，對照電壓與光線條件
光線分類：用自己的資料訂規則	算分界、執行程式、說明判斷的限制
Q3：交換兩顆固定電阻	接線、量測、計算電壓及畫電流
Q5：按一下，記錄一筆光線	完成接線與程式，解釋資訊流

這個作品使用電表、光敏模組與按鈕，在電腦保存光線資料。

量測欄記下自己的數字與條件；尚未量測的欄位留白，不抄示例當實測。

開啟本週的程式

本週 Ans 隨附 programs 資料夾。以下路徑從本週資料夾開始找；每次只開當段指定的檔案。

在 Arduino IDE 選「檔案 → 開啟」，開啟資料夾內的同名 .ino，再另存自己的工作副本。不要把不同範例貼在同一支程式。

用途	開啟的檔案 (位於同名子資料夾)
GPIO 電壓切換	week03_gpio_voltage_cycle.ino
光敏原始讀值	week03_ky018_raw.ino
光線分類	week03_light_classifier.ino
按一下取一筆	button_light_capture.ino
A：三筆取樣	three_light_samples.ino
B：遮光計數	shade_counter.ino
OLED 通訊檢查	week05_i2c_check.ino
C：光線觀測	light_observer.ino
D：光線快照	light_snapshot.ino

各段會說明接線、程式庫及需修改的設定。完整程式在檔案中；本冊只保留必要的關鍵片段與原理解釋。

改線前先斷電。確認腳位與供電後才依當段步驟上傳；換程式時重新核對設定，不沿用尚未確認的值。

清空接線，準備量電壓

GPIO4、GPIO5 這週要換用途，不能沿用計數器接線。

1. 拔掉開發板的 USB 線，移除電池及其他外部供電，確認開發板上標示 PWR 的電源指示燈熄滅。
2. 將萬用電表的旋鈕轉到 OFF (關機)，移開表筆。拍下舊作品接法後，拆除開發板所有外接線。
3. 拿空的 400 孔麵包板。板子放在乾燥、不導電桌面，不直接插進麵包板。
4. 按鈕先收起來，完成電表、感測器與分壓實作後，再接回按鈕取樣。

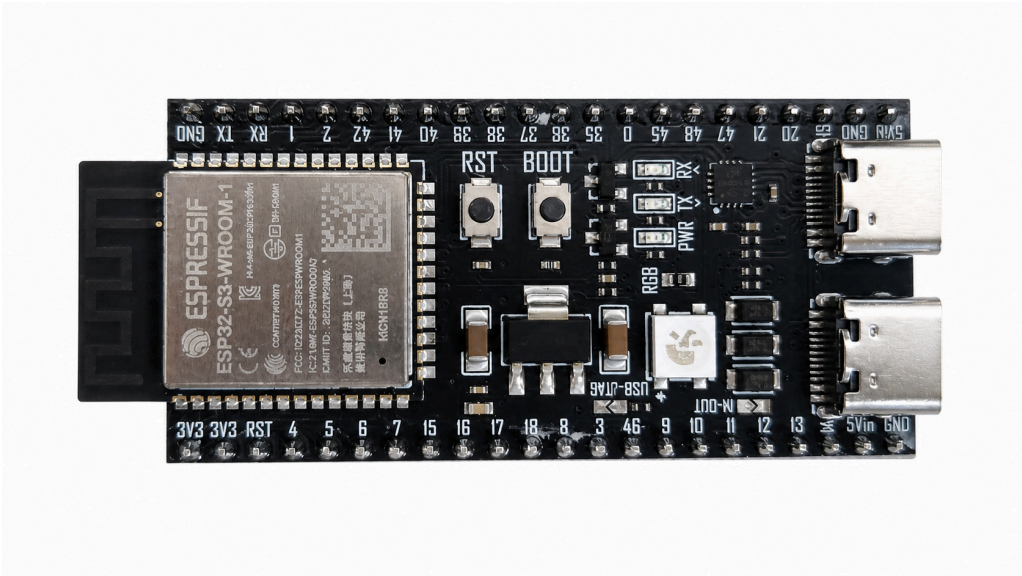
現在拿出來	數量
圖示 ESP32-S3 開發板、USB 資料線、筆電	各 1
萬用電表、400 孔麵包板	各 1
公對母 / 公對公 / 母對母杜邦線	各數條
KY-018 光敏模組、不透光的紙	各 1
電阻包中的 1 k Ω 、10 k Ω	各 1 顆
Week2 使用過的四腳按鈕	1 顆

以後每次改線：先移開表筆 → 拔 USB 與其他電源 → 確認 開發板 PWR 電源燈熄滅 → 電表旋鈕轉到 OFF → 才移線。關閉 Serial Monitor 不等於斷電。

本週三個用途：3V3 給小型感測模組供電；GPIO5 暫時輸出高 / 低電壓；GPIO4 稍後讀光敏訊號。這些用途不是同時接上舊按鈕。

只找四個腳位

天線在左，USB 在右。看印字，不是數第幾根針。



課堂實物正面：YD-ESP32-S3 Type-A V1.5 · N16R8。

開發板接腳旁的印字	這張照片的位置	用途
GND	上排最左端，靠天線	共同參考點
3V3	下排最左端	約 3.3 V 電源
5	下排，在 4 與 6 之間	稍後輸出電壓
4	下排，在 RST 與 5 之間	稍後讀光敏訊號

不要接開發板上標示 5Vin 的接腳。這片板的 IN-OUT 焊盤未橋接時，該腳不是 USB 的 5 V 輸出；它也不是 GPIO5。本週不焊接、不橋接焊盤。

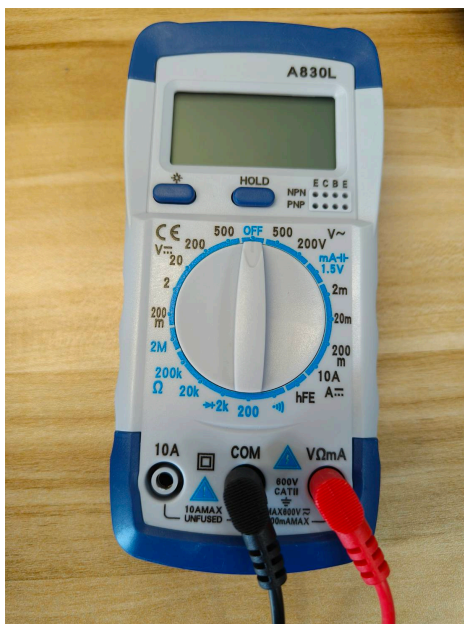
本稿接法對應圖示板型。不同板型、看不清印字或課程尚未確認 GPIO 用法時，保持斷電，一起核對。照片不能證明所有同名商品相同。

本段量 3V3 對 GND 的電壓，GPIO4、GPIO5 保持未接線。

黑筆、紅筆、旋鈕各放哪裡？

這次量電壓，不是量電流，也不是聽通斷聲。

表筆一端是插進電表的插頭，另一端是碰測量位置的金屬筆尖。V 是電壓單位「伏特」；旋鈕的 20 V 檔是量測範圍，不是輸出電壓。



課堂 A830L 實物照片。照片旋鈕在 OFF；量測前仍須自己選檔。

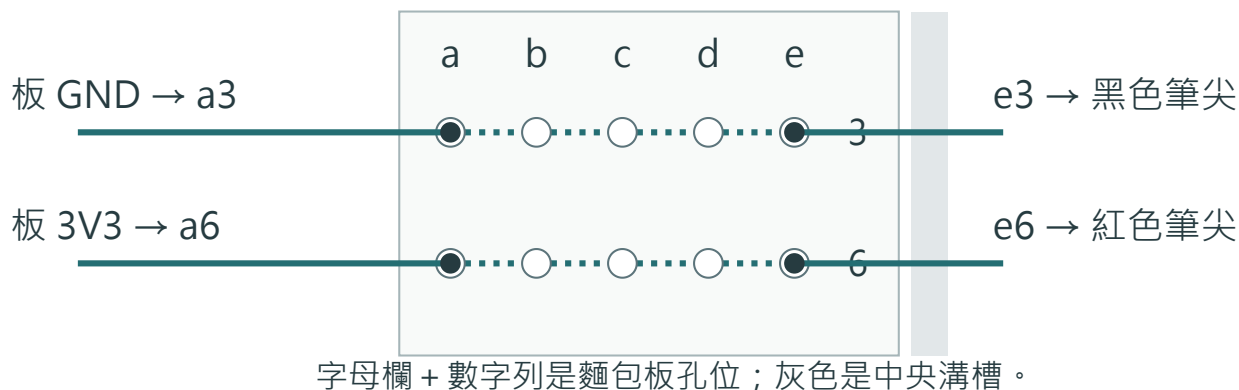
位置	這次設定
黑色表筆的插頭	插進電表下方中間、標示 COM 的孔
紅色表筆的插頭	插進電表下方右邊、標示 VΩmA 的孔
電表旋鈕	直流電壓 20 V：找 V 與實線 / 虛線符號，不是 V~
電表下方標示 10A 的插孔	電流量測用，本週保持空著

電表 COM 插孔是共用參考端。黑色表筆的筆尖碰到開發板 GND 腳的延長導線，兩者才連起來。插孔旁的 10A 表示電流量測用途；A 是安培，不是麵包板孔號。

本週完全不使用 A / mA 電流檔，也不量插座或市電。表型不同時看自己的符號與說明，不照照片角度猜。先設好，再將表筆放旁邊。

兩條線，把電源位置引出來

麵包板只讓表筆比較好碰，不會自己製造電源。



圖中 a3、e6 等是麵包板孔位：字母指欄、數字指列。3V3、GND 是開發板接腳名稱。延長線一端插麵包板，另一端金屬露出，供表筆筆尖碰觸。

先後	起點 → 終點	杜邦線怎麼接
1	ESP32 GND → a3	母頭套排針，公頭插孔
2	ESP32 3V3 → a6	母頭套排針，公頭插孔
3	e3 → 一條公對公線的自由端	黑色筆尖稍後碰這個自由端
4	e6 → 另一條公對公線的自由端	紅色筆尖稍後碰這個自由端

同列 a~e 相通。a3 與 e3 是同一位置的延伸；a6 與 e6 是另一組。第 3 列和第 6 列不可用線直接相連。兩個自由金屬端要分開固定。

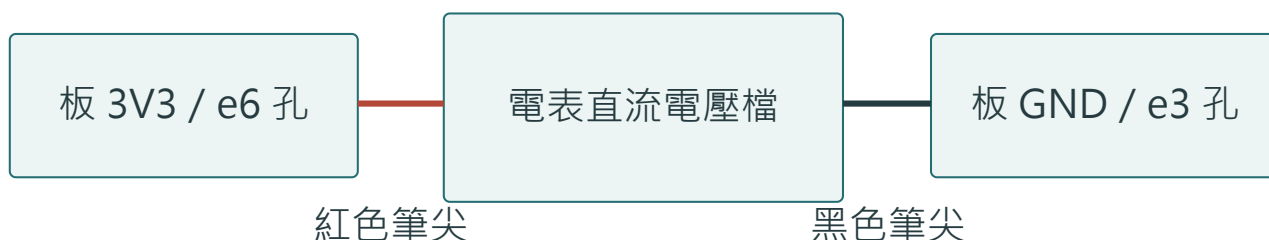
一起核對表格及列號，確認沒有其他線或元件跨接第 3、6 列。若做通斷複查，只能在拔 USB 時做；供電與 GND 持續蜂鳴時保持斷電排查。不蜂鳴也不是完整安全保證。

表筆不要硬塞進麵包板。不要讓延長線金屬端互碰，也不要一邊通電一邊整理線。

黑筆固定，紅筆只碰一個端點

先猜：3V3 相對 GND，會接近 0 V 還是 3.3 V？

1. 電表旋鈕確認為直流電壓 20 V 檔；用 USB 線連接電腦與開發板背面標示 COM 的 USB 接頭。這個 COM 接頭不是電表 COM 插孔。
2. 黑色筆尖碰插在麵包板 e3 孔的延長線金屬端。
3. 紅色筆尖碰插在麵包板 e6 孔的延長線金屬端，不碰旁邊排針。
4. 接觸穩定後記錄完整畫面與單位，再移開紅筆。



e6、e3 是麵包板孔位；示例顯示約 +3.3 V。

黑色筆尖測點	紅色筆尖測點	電表檔位	預期	我的結果
e3 · GND	e6 · 3V3	直流 20 V	約 +3.3 V	___ V

預期為接近 +3.3 V 的正值，不要求每片板或每支電表完全相同；記錄自己看到的數字。

如果是負數：先移開表筆，核對紅黑測點，不交換板子的供電線。**如果接近 0 V：**移開表筆、斷電，核對 USB、PWR、3V3 印字與列號。

電壓是比較兩個位置

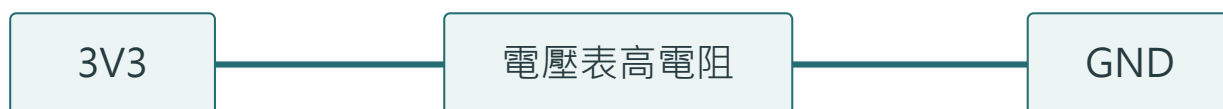
剛才的 +3.3 V，表示紅色筆尖位置比黑色筆尖位置高約 3.3 V。

電表在直流電壓檔顯示：紅色筆尖位置的電位 – 黑色筆尖位置的電位。本週把 GND 當共同的 0 V 參考。

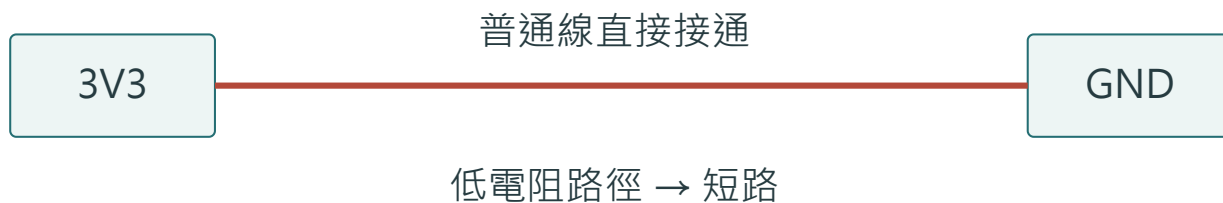
紙上情境，不必重新接線	計算	顯示
紅色筆尖碰開發板 3V3 腳、黑色筆尖碰 GND 腳	$3.3 - 0$	+3.3 V
紅色筆尖碰開發板 GND 腳、黑色筆尖碰 3V3 腳	$0 - 3.3$	-3.3 V
兩支筆尖碰同一個 3V3 接點	$3.3 - 3.3$	0 V

第三列不表示板子沒電，只表示兩個測點的電位相同。GND 也不是讓電流憑空消失的地方，它接回電源回路。

正常量測：



不要實作：



為什麼電表跨接 3V3 和 GND 不等於短路？電壓檔內部有很大的輸入電阻，只取很小的電流。普通杜邦線的電阻很低，不能拿來代替電壓表。

上圖只是比較電路，不實作直接短接。若把電表切到電流檔，內部路徑不同，就不能照剛才的方式跨接電源。

換一支會切換電壓的程式

上週 GPIO5 是減一按鈕；現在先讓它成為輸出。

1. 移開表筆、拔 USB、電表旋鈕轉到 OFF。從開發板拔下外接線；麵包板的延長線可以留著。
2. 在 Arduino IDE 選 **File → Open**，開啟隨課程提供的 `programs/week03_gpio_voltage_cycle/week03_gpio_voltage_cycle.ino`。完整檔案放在本週 programs 資料夾。
3. 找到 `PIN_TEST_OUTPUT = -1`。本稿示範接點是 **GPIO5**；只有課前已確認使用圖示板型與這個接點時，改為 5。未確認保留 -1，不猜其他數字。
4. 只用 USB 線連接電腦與開發板上標示 COM 的 USB 接頭，依下表選設定，按 IDE 左上方右箭頭 **Upload (上傳)**。

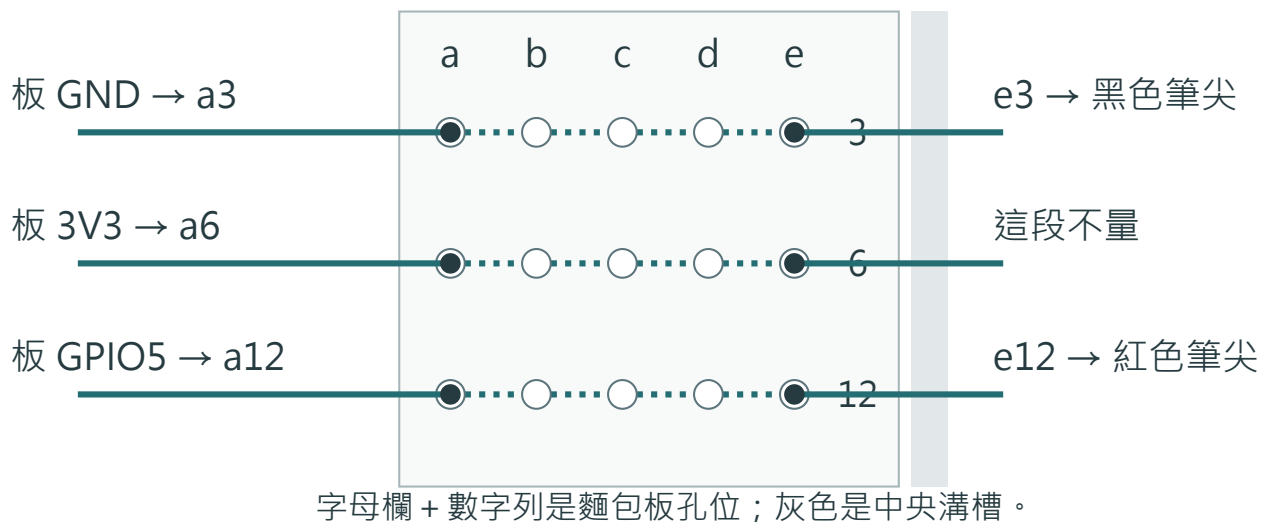
IDE 位置	本課圖示板設定
Tools → Board	ESP32S3 Dev Module
Tools → Port	選電腦辨識開發板後新增的連接埠，如 COM7；號碼依電腦而異
Flash Size / PSRAM	16MB / OPI PSRAM
USB CDC On Boot / USB Mode	Disabled / Hardware CDC and JTAG
Upload Mode / Upload Speed	UART0 / Hardware CDC / 115200
Flash Mode / Partition Scheme	QIO / 16M Flash (3MB APP/9.9MB FATFS)

在 Arduino IDE 開 **Tools → Serial Monitor**，把視窗的通訊速率選單設為 **115200 baud**。這是電腦與程式的文字通訊設定。預期看到 `startup=LOW`，之後約每 10 秒出現 LOW 或 HIGH，表示 GPIO 的低 / 高電壓狀態。

沿用 Week 2 的 IDE 安裝與 Espressif 套件。尚未安裝者依 Week 2 安裝段完成設定。上傳後先拔 USB，再依第 24 頁接 GPIO5 量測線。

加一條 GPIO5 線，量它的電壓

沒有按鈕、LED 或其他負載；GPIO5 只接電表量測。



起點 → 終點	現在的作用
GND → a3 ; e3 → 黑筆延長端	恢復共同參考點
3V3 → a6 ; e6 延長端分開收妥	恢復電源測點，這段不量它
GPIO5 → a12	新增程式控制的測點
e12 → 新的公對公延長線	紅色筆尖稍後碰這個端點

每條外接線照表恢復，特別確認 GPIO5 不是 5Vin。第 12 列不能直接接到第 3 列或第 6 列。

第 12 列是 GPIO5 的量測接點。導線把 GPIO5 接到 a12，同列的延長線讓紅表筆量到這個腳位相對 GND 的電壓。

先確認舊按鈕已拆除。GPIO5 若輸出 HIGH 時仍被按鈕直接接 GND，會造成過大電流的風險。不要沿用計數器接線。

LOW、HIGH，電表真的不同嗎？

先猜：哪個階段接近 0 V？哪個接近 3.3 V？

1. 確認接線後連上開發板上標示 COM 的 USB 接頭，在 Arduino IDE 開 Serial Monitor，通訊速率選 115200 baud。
2. 電表旋鈕選直流電壓 20 V 檔。黑色筆尖碰麵包板 e3 孔延長線的金屬端；紅色筆尖碰 e12 孔延長線的金屬端。
3. 看 phase=LOW 或 phase=HIGH，等讀值穩定再記錄。不要把切換瞬間的跳動值當穩態。

次數	畫面 phase 欄 (輸出狀態)	預期電壓	實測電壓
1	LOW	約 0 V	___ V
1	HIGH	約 3.3 V	___ V
2	LOW	約 0 V	___ V
2	HIGH	約 3.3 V	___ V
3	LOW	約 0 V	___ V
3	HIGH	約 3.3 V	___ V

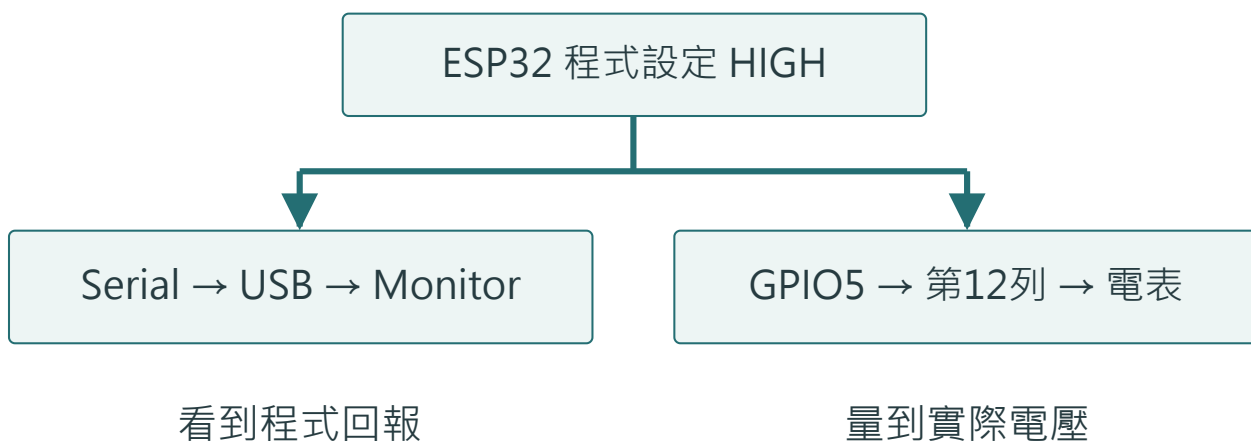
再看一次啟動：移開兩支表筆，按開發板上標示 RST 的重新啟動按鈕，保存畫面重新出現的 startup=LOW 文字。這裡只看啟動紀錄，不重做整張電壓表。

``expected_voltage=near_3.3V`` 是程式寫出的預期，不是 ESP32 已幫你量過。電表穩態讀值也不能證明 Reset 瞬間完全沒有短脈衝。

完成後拔 USB、電表旋鈕轉到 OFF。

一個命令，兩種證據

Serial 說「我要求 HIGH」；電表說「我量到幾伏特」。



程式中的動作	對應剛才哪件事
<code>pinMode(..., OUTPUT)</code>	指定 GPIO 用來輸出，不再讀按鈕
<code>digitalWrite(..., HIGH/LOW)</code>	要求輸出高 / 低準位
<code>millis()</code> 的時間差達 10000	<code>millis()</code> 記開機後毫秒；相差 10000 ms 就是約 10 秒
<code>Serial.printf(...)</code>	把命令與次數送到電腦

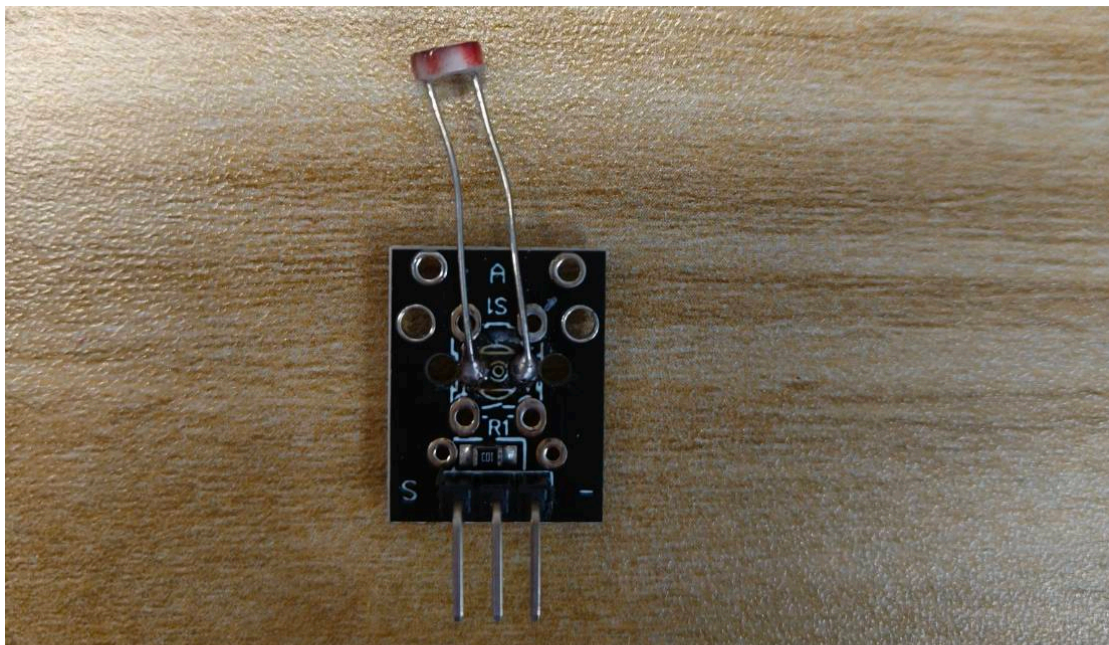
3V3 負責供電；GPIO5 負責輸出訊號。剛才程式讓 GPIO5 在高、低電壓間切換。即使量到約 3.3 V，也不能把 GPIO 當一般電源帶動馬達或模組。

自己回答：如果 Monitor 顯示 HIGH，電表卻是 0 V，能說程式完全正確、接線也正確嗎？先斷電查腳位、列號和上傳版本，不能只相信文字。

下一段要讀光敏訊號。保持斷電，拆除 GPIO5 → a12 與 e12 延長線；3V3、GND 兩列稍後繼續使用。

找到會感光的元件和三支腳

模組先放桌上，不插麵包板，不接 USB。



實物照片局部放大，原圖未修改：圓形感光元件朝上，三支排針朝下。

照片中的排針	已有實物紀錄的用途
左邊，獨立 S 標示旁	訊號，稍後量它的電壓
中間，沒有獨立供電標示	本課已核對模組的供電端
右邊，獨立 — 標示旁	接 GND

比對自己的絲印和照片。板上 **A**、**S1**、**R1** 是元件區標示，不能直接當排針名稱。三組路徑須在斷電下量測。

照片朝向改了，左右也會改。不同模組不能只照左右猜供電。本稿只適用已確認「中間到 S 是固定電阻、S 到 — 是光敏電阻」的接法；結果不符就不供電。

不接電，也能先看阻值變化

這裡改用 Ω 檔；ESP32、USB、電池都不能接在模組上。

Ω 是電阻單位「歐姆」；k 表示一千，所以 $1\text{ k}\Omega = 1000\ \Omega$ 。200k Ω 是量程，不是模組一定有 200 k Ω 。

1. 黑色表筆插頭留在電表 COM 插孔，紅色插頭留在 V Ω mA 插孔。電表旋鈕改選電阻區的 200k Ω 檔。
2. 黑、紅表筆的筆尖各碰下表指定的模組排針，手只握絕緣部分。固定接觸後，用紙遮住感光面，不碰動排針。
3. 同一組測點依序比較「室內光 → 遮光 → 移開紙」。換測點時也保持模組不接電源。

光敏模組上的兩個測點	室內光	遮光	移開紙
中間供電腳 ↔ S 訊號腳	___ k Ω	___ k Ω	___ k Ω
S 訊號腳 ↔ 標示 - 的接地腳	___ k Ω	___ k Ω	___ k Ω
中間供電腳 ↔ 接地腳	___ k Ω	___ k Ω	___ k Ω

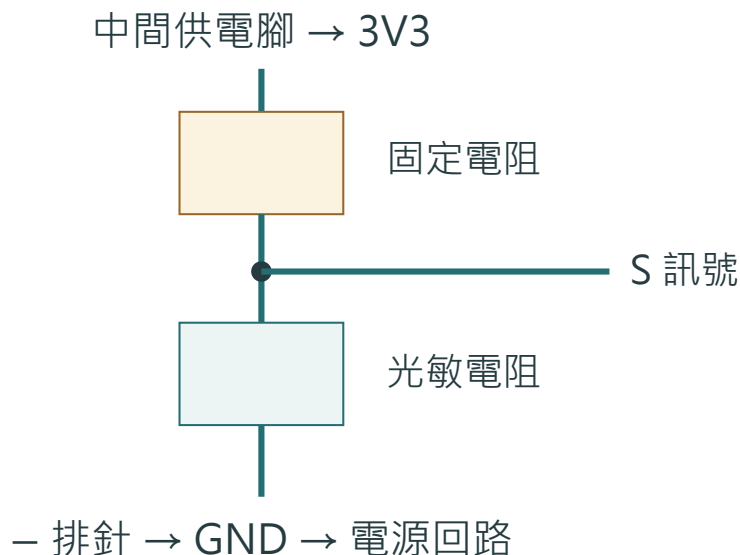
記下**檔位、完整小數點與單位**。200k 檔例如顯示 10.0，通常是 10.0 k Ω ，仍以自己的表型說明為準。若超出量程顯示 1 / oL，不要寫成 1 Ω ；提高量程或保存「超量程」。

先看變化，不求精密驗證：本課既有接法預期中間 ↔ S 比較固定；S ↔ - 隨光線改變。沒有變化先查遮光與接觸，不交換供電腳來試。

量完電表旋鈕轉到 OFF，移開表筆。

光敏模組裡有兩個電阻

固定電阻的阻值大致不變；光敏電阻的阻值會隨光線改變。



上面的固定電阻：接在中間供電腳與 S 之間。**下面的光敏電阻：**接在 S 與 — 之間。S 是兩者中間的位置，不是新的電源。

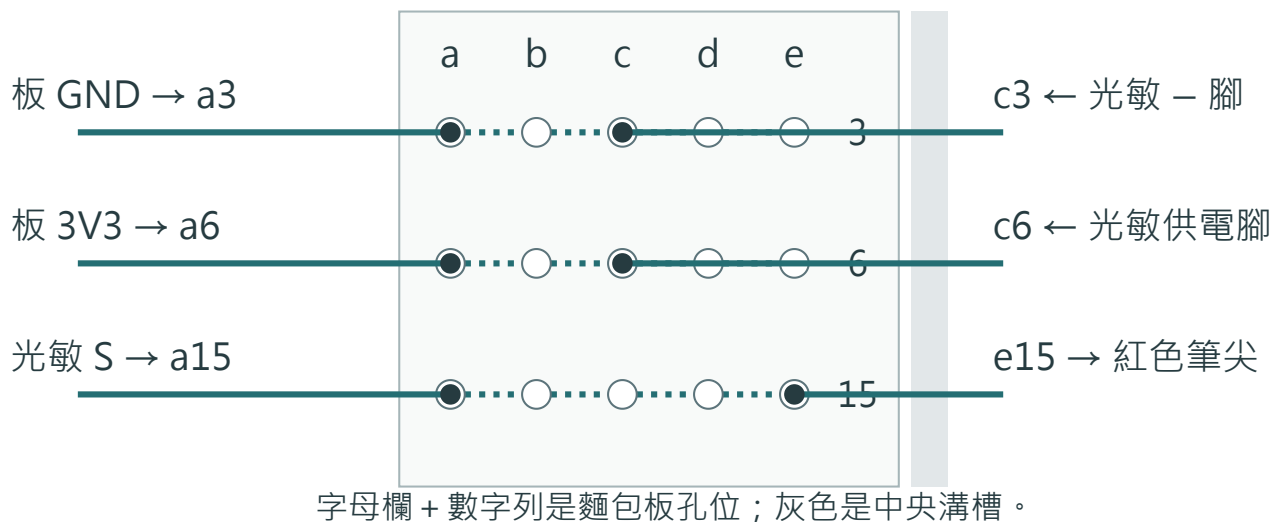
剛才量的路徑	這張圖的對應
中間 ↔ S	上面一段
S ↔ —	下面一段
中間 ↔ —	兩段串聯的總路徑

同樣光照與接觸下，總阻值應大致接近兩段相加。三次量測若光照變了、手碰到金屬或量程不同，不能把差異當成精密驗證通過。

S 相對 GND 的電壓取決於兩段電阻的比例。光敏電阻改變時，這個中間位置的電壓也會改變。

先接供電，再把 S 引出來

這一步只給模組供電。S 還不接 GPIO4。



開發板 / 模組接腳 → 麵包板孔位	說明
ESP32 GND → a3 ; 3V3 → a6	保留前段兩條電源線
光敏模組 中間供電腳 → c6	用母頭套模組腳，公頭插孔
光敏模組 - → c3	接到同一個 GND
光敏模組 S → a15	只是引出訊號，不接 GPIO
e15 → 新的公對公延長線	紅表筆的訊號測點
e3 → 黑筆延長線	沿用 GND 測點

GPIO5 與第 12 列的線已拆除。e6 電源延長線這段不用，斷電時拆下，避免紅筆量錯。模組放不導電桌面，不把三支腳插在麵包板同一個五孔組。

一起核對中間腳、S、- 及列號。S 不直接短接 GND；GPIO4 暫時空著。只用 3V3，不用 5Vin、5V 或其他電池供電。

遮住之後，電壓升還是降？

先猜一次，再用自己的讀值回答。

1. 電表旋鈕選**直流電壓 20 V 檔**，不能留在上一段的電阻 Ω 檔。
2. 連上開發板上標示 COM 的 USB 接頭。黑色筆尖碰麵包板 e3 孔延長線的金屬端；紅色筆尖碰 **e15** 孔延長線的金屬端，不是 e6 孔。
3. 不移動模組，記錄室內光；用不透光紙遮住感光面，再記錄；移開紙，最後再記一次。

條件	預期觀察	模組 S 腳對開發板 GND 腳電壓
室內光	可穩定讀取一個電壓	___ V
原位置遮光	電壓與室內光有差異	___ V
移開紙	接近同位置室內光讀值	___ V

我的觀察：遮光後電壓 _ _ ；移開紙後 _ _ 。板子、檯燈、紙的位置與表筆接觸是否都固定？
_ _ 。

不憑 KY-018 商品名稱判斷方向。對圖示電路，下方光敏電阻變大時，S 電壓會升高；若實測相反，先保留結果，斷電核對模組內部接法與測點。

讀值超出 0 ~ 3.3 V、負值或異常發熱：先移開表筆、斷電，查供電與測點。正常觀察完成後也先移開表筆、拔 USB、電表旋鈕轉到 OFF。

如果模組 S 腳電壓一直等於供電電壓，移開表筆，確認紅色筆尖碰的是麵包板 e15 孔延長線，不是 e6 孔的電源延長線；接點鬆脫要斷電後處理。

把 S 的電壓變成電腦上的數字

先換程式，再把訊號接入 GPIO4。不要沿用 GPIO 輸出程式。

1. 保持斷電，從開發板拔下外接線。麵包板與模組間的線可保留，沒有電源供應。
2. IDE 選 **File** → **Open**，開啟 `programs/week03_ky018_raw/week03_ky018_raw.ino`。
3. 找 `PIN_LIGHT = -1`。課前已確認本稿的 GPIO4 接法時改成 4，否則保留 -1。不要沿用上一支程式的 5。
4. 將 `DEVICE_ID = "CHANGE_ME"` 改成自己的組別代碼，例如 "G01"。不要放姓名或學號。
5. 只將開發板上標示 COM 的 USB 接頭連到電腦，依第 23 頁重新核對板型與 Port，按 Upload。

Serial Monitor 的通訊速率選 115200 baud。看到 `status=ready` 及 `interval_ms=500`，表示程式已開始準備取樣，不代表線已接對。

```
device=G01 ... adc_bits=12 interval_ms=500
```

這是欄位示意，不是本次實測。尚未接模組的輸入可能亂跳，**現在的 raw 不拿來當光線資料**。

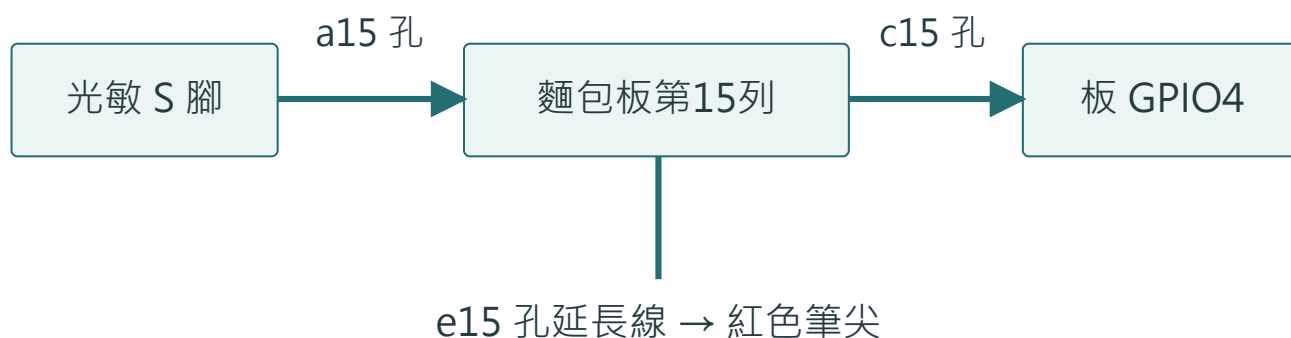
ADC 把接腳電壓轉成整數，這個結果叫原始讀值 raw。程式用 `analogRead` 取得它；例如 `light_raw=310` 就是本次轉換得到 310。

完成後拔 USB，依第 33 頁接回模組電源與 S 訊號。

多一條線：c15 接到 GPIO4

第 15 列是同一個 S 訊號，電表和 ESP32 可以看同一位置。

新增訊號線；模組供電接法保持不變



黑色筆尖碰 e3 孔延長端；e3、c3 通開發板 GND。

接腳與麵包板孔位	現在要做
ESP32 GND → a3 ; 3V3 → a6	接回剛才拔下的電源線
光敏模組 – → c3 ; 中間 → c6 ; S → a15	維持上一段接法
c15 → ESP32 GPIO4	新增公對母線；不是 GPIO5
e15、e3 的兩條延長線	分開固定，供稍後量 S / GND

a15、c15、e15 是同一五孔組的三個入口。不是三個不同訊號，也不需要再把 S 直接接一條線到 GND。

GPIO5 保持空著；4 不是 3V3。所有訊號只能在已確認的安全電壓內。ESP32-S3 GPIO 不接受 5 V 訊號；本週模組以 3V3 供電。

對照接點後接上開發板上標示 COM 的 USB 接頭，開 Monitor，通訊速率選 115200 baud。先固定模組、遮住、移開一次，看數字是否改變，再開始保存資料。

把 Monitor 的紀錄存成文字檔

保持接線與感測器位置不變。室內光與遮光各存十筆，分成兩份檔案；檔名註明條件，不把試打文字混入量測資料。

1. Serial Monitor 的通訊速率選 115200 baud，等數行完整文字出現。資料仍會繼續來，不用拔 USB 暫停。
2. 在 Monitor 關閉自動捲動，或捲回要保存的位置。用滑鼠從第一行開頭拖到最後一行結尾，按 **Ctrl+C**。
3. 開 Windows「記事本」，按 **Ctrl+V**。確認每行有完整的 `sample`、`raw` 和時間，不是只有截圖或最後半行。
4. 最上方加一行條件，例如「日期 / group01 / 室內光 / 同一桌面 / 剛才未重開機」。
5. 選「檔案 → 另存新檔」，檔名例如 `week3_group01_indoor.txt`，編碼選 **UTF-8**。回 Monitor 繼續觀察。

兩份量測檔案	應有的內容
<code>week3_group01_indoor.txt</code>	同輪室內光十筆完整行與條件
<code>week3_group01_shade.txt</code>	同接法遮光十筆完整行與條件

若拖選漏行，就重新選取那一段；不要補寫不存在的輸出。重開機或換位置後另存一輪，不能把不同輪資料混在一起。後面各週也用同樣方法保存 Serial 紀錄。

室內光十筆，遮光十筆

固定位置與接線，只改變遮光條件，才能比較兩組讀值。

1. 模組、桌燈、電源、接線位置不變。先預測遮光後 raw 比室內光大還是小：_____。
2. 室內光穩定後，從 Monitor 保存連續 **10 行**，標記「室內光」。保留序號、時間及 raw。
3. 用同一張紙在原位置遮住感光面。穩定後另存連續 **10 行**，標記「遮光」。
4. 每種條件也量一次模組 S 腳對開發板 GND 腳的電壓：電表旋鈕選直流電壓 20 V 檔，紅色筆尖碰麵包板 e15 孔延長端，黑色筆尖碰 e3 孔延長端。先量電壓，再存文字紀錄；兩者不是同步量測。不要帶電移線。

```
device=G01 sample=21 uptime_ms=10500 light_raw=310
device=G01 sample=22 uptime_ms=11000 light_raw=314
```

上面是**預期格式示例**，**不是實測**。只讀最後的 light_raw 作光線原始值；sample 是序號，uptime_ms 是開機後毫秒。

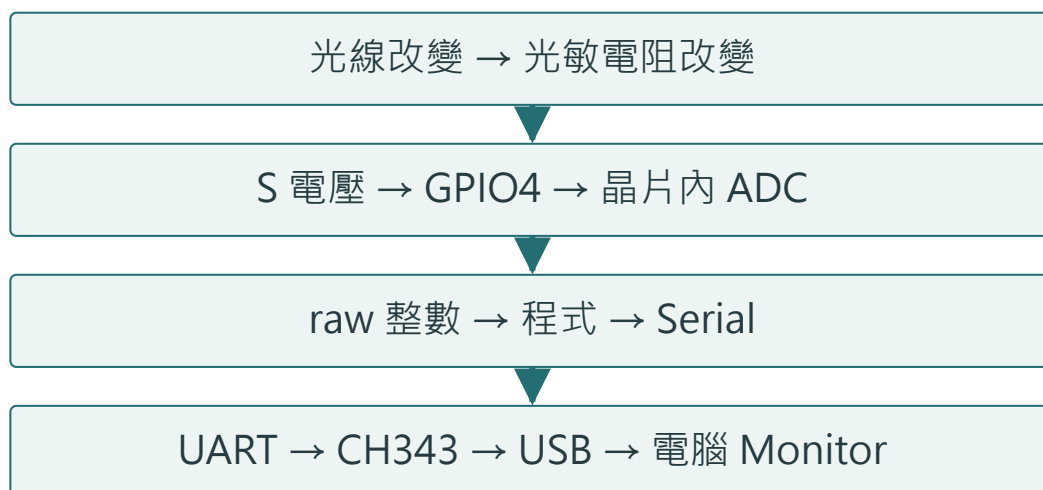
條件	10 筆最小值	10 筆最大值	電表
室內光	_____	_____	_____ V
遮光	_____	_____	_____ V

保存全部 **20 行**，表格只是摘要。500 ms 是 0.5 秒；10 筆有 9 段間隔，首尾約 4.5 秒。若中途重開機或移位置，另開一輪，不混算。

若 raw 一直顯示 0 或 4095，先依上方測點量電壓，再斷電查線。這兩個讀值是原始讀值範圍的兩端，可能來自超出 ADC 量測範圍或接線問題，不能只憑 raw 判斷斷線。

ESP32 怎麼把電壓變成數字？

ADC 把 GPIO4 相對 GND 的電壓轉成整數；程式把它記為 raw。



UART 與 CH343 沿用 Week 2 的文字傳送路徑；箭頭是資訊傳遞，不是新增接線。

本程式設定 **12-bit**：4096 個整數，從 0 到 4095。raw=700 就是本次轉換得到 700；相同設定、未飽和時，較高電壓通常得到較大讀值。

程式中的 `ADC_11db` 設定用來選擇 ADC 量測範圍；文件列此板晶片約可量 0 ~ 3.1 V。超出可轉換範圍時，raw 可能卡在 0 或 4095，稱為飽和。raw=700 不能直接當 0.700 V 或 700 lux（照度單位）；4095 也不保證正好等於 3.3 V。

現在可以說	不能直接說
此輪遮光後，我的 raw 上升 / 下降	所有 KY-018 都同方向
電表與 raw 在此輪大致同向改變	我已精確校正 ADC 電壓
接法相同時，數字可用來比較	數字就是照度 lux

一起解釋：用自己的 20 行資料回答「raw 變大代表比較亮嗎？」先說哪個條件，再指出兩組範圍，不能只回答是或不是。

模組 — 與板子 GND 是共同參考；訊號送到 GPIO4。ADC 內部完成比較，不是叫你在板外用線把 GPIO4 和 GND 短接。

實測讀值與電壓

Q1 · 你的作品在兩種光線下各讀到多少？

沒有固定數字答案。沒遮光、遮光各記三筆 raw，與本組畫面一致。固定位置、光源與接線，只改變是否遮住感測器；不可要求全班數字相同，也不能用理論值代替實測。

操作見 p32 ~ 35。其中室內光、遮光各存十筆，Q1 可各取其中連續三筆。兩組讀值留作 B 的判斷依據。

讀值一直停在 0 或 4095 時，先保存觀察，再查訊號電壓、量測範圍與接線；不能只憑端點說「一定遮光」或「一定斷線」。

Q2 · raw=800 能直接寫成 0.8 V 嗎？

不能。800 是 ADC 將訊號電壓轉換後的整數，沒有伏特單位；換成電壓需要對應 ADC 設定的換算與校正資料，不能直接除以 1000 就當成伏特。相關原理見 p36。

先畫兩段範圍，再選分界

用剛才資料，不重新收一樣的二十筆。

教學假資料；raw 無單位，不是 V



示意位置非等比例。兩段之間的值沒有基準資料。

上圖是**教學假資料**：室內光 300 ~ 320，遮光 900 ~ 920。兩段沒有重疊，中間可以選一個分界：

分界 = (室內最大值 320 + 遮光最小值 900) ÷ 2 = 610。在這個方向下，raw ≤ 610 判室內光，raw > 610 判遮光。

你的資料狀態	接下來怎麼做
遮光整段比室內光大	用室內最大值與遮光最小值求中點
遮光整段比室內光小	用遮光最大值與室內最小值求中點；方向反過來
重疊，或兩段碰到同一數字	目前無法用這個分界分清，先保存結果，不硬設

填自己的四個值：室內最小 ____、室內最大 ____、遮光最小 ____、遮光最大 ____。我的分界 ____；遮光是較大 / 較小 / 還無法分開。

最小 / 最大值是這一輪的範圍，不是未來永遠的界線。不要刪掉不合預期的數字，只為了讓範圍分開。

讓程式印出「室內光 / 遮光」

這次 GPIO4 仍是輸入，模組接法不變。

先移開表筆、拔 USB、電表旋鈕轉到 OFF，從開發板拔下外接線。關閉上一個 Monitor，IDE 開啟 `programs/week03_light_classifier/week03_light_classifier.ino`，只填下面這些設定：

程式名稱	填什麼
PIN_LIGHT	與上一支原始讀值程式相同、已確認的腳位
DEVICE_ID	同一組別代碼
INDOOR_MIN、INDOOR_MAX	自己室內光十筆的最小、最大值
SHADE_MIN、SHADE_MAX	自己遮光十筆的最小、最大值

分界與方向由程式算，不必另把 610 抄進程式。只將開發板上標示 COM 的 USB 接頭連到電腦，依第 23 頁設定後 Upload。拔 USB，依第 33 頁接回光敏電路；再接 USB，Monitor 的通訊速率選 115200 baud。

```
status=ready ... threshold=610 shade_higher=true
... light_raw=310 ... class=INDOOR label=室內光
```

上面沿用前頁假資料示範格式，**你的分界應來自自己的資料**。若出現 `calibration_missing_or_overlap`，表示有缺值、不合範圍或重疊；回前頁查四個設定，不拆線試錯。

既有分類程式保守排除 `raw=0、4095` 的基準。這是課堂規則，不代表端點讀值一定是斷線。尚未取得可分開的基準時，保留原始資料與原因就好，不偽造通過。

用新資料檢查分類結果

驗證就是把程式的判斷和實際條件比較；這次各取五筆新資料。

1. 先在相同位置回到室內光，保存 **5 筆新紀錄**。
2. 再以相同方式遮光，保存 **5 筆新紀錄**。
3. 每筆比較實際光線與紀錄的 `label`（分類文字），也保留 `quality`（判斷適用情況）和 `reason`（原因）欄位。

真實操作條件	新資料筆數	標籤相符	不相符 / 未判定
室內光	5	——	——
遮光	5	——	——

例：若 10 筆有 8 筆標籤相符，可以說「這次、這個位置的 10 筆有 8 筆相符」。不能說整個系統準確率永久 80%，也不能把短時間連續取樣視為十種環境。

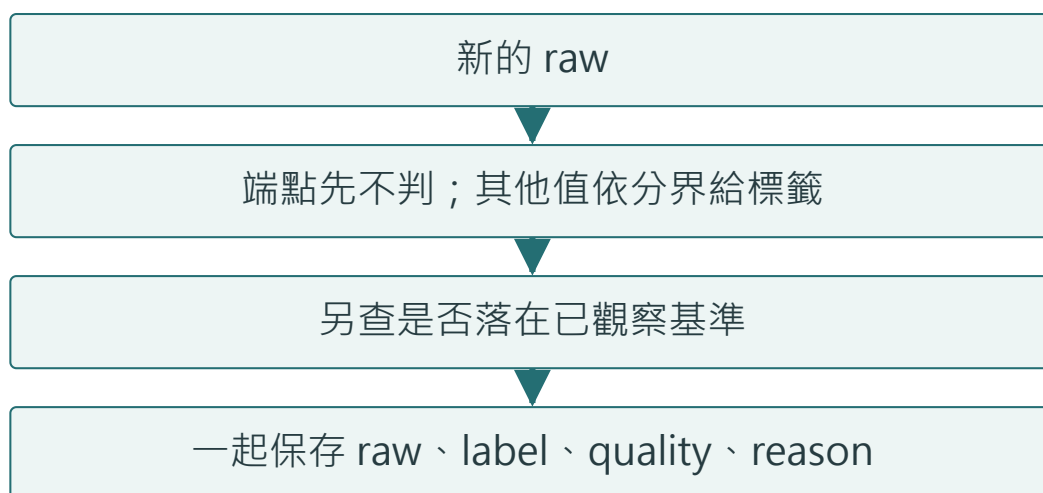
若全部都判同一類：先對照原始值與四個範圍，確認條件、上傳版本、模組位置與組別資料。不要先交換電源線。

同一個 `raw` 落在兩組範圍之間時，程式雖然印了標籤，基準紀錄中是否真的出現過這個數字？

看 raw，也看為什麼這樣判

沿用假資料：室內 300 ~ 320、遮光 900 ~ 920、分界 610。

新 raw	畫面分類	quality / reason 欄
310	室內光	relative_only：在已觀察的基準內，仍只是本地相對判斷
700	遮光	suspect / between_baselines：落在兩段之間，沒有基準支持
1000	遮光	suspect / outside_observed_span：超出曾觀察的總範圍
0 或 4095	未判定	suspect / adc_endpoint：端點先不分類，不等於證明斷線



這些英文是紀錄的欄位：raw 是 ADC 原始整數，label 是分類文字，quality 標記判斷的適用情況，reason 說明和基準的關係。例如 700 雖判遮光，仍標記缺少基準支持。

討論：假設剛才遮光較大，現在換位置卻較小，先查哪兩件事？先保存兩輪，不合併；再查接法 / 模組是否相同，以及光線、遮法和接觸是否改變。找到差異前不能套用舊規則。

下一段斷電後拆下光敏模組。用兩顆固定電阻，把「為什麼電壓會變」拆成看得見的電路。

拿出 1 k Ω 和 10 k Ω

先量實物，再接電路。不是只看包裝就假設一定正確。

移開表筆，拔 USB，電表旋鈕轉到 OFF。拆下 光敏模組 的供電、GND、S 線，以及 GPIO4 線與第 15 列延長線。保留 GND → a3、3V3 → a6；板子仍斷電。



已購電阻包的實物與手寫標示。本段只取 1 k Ω 、10 k Ω 各一顆。

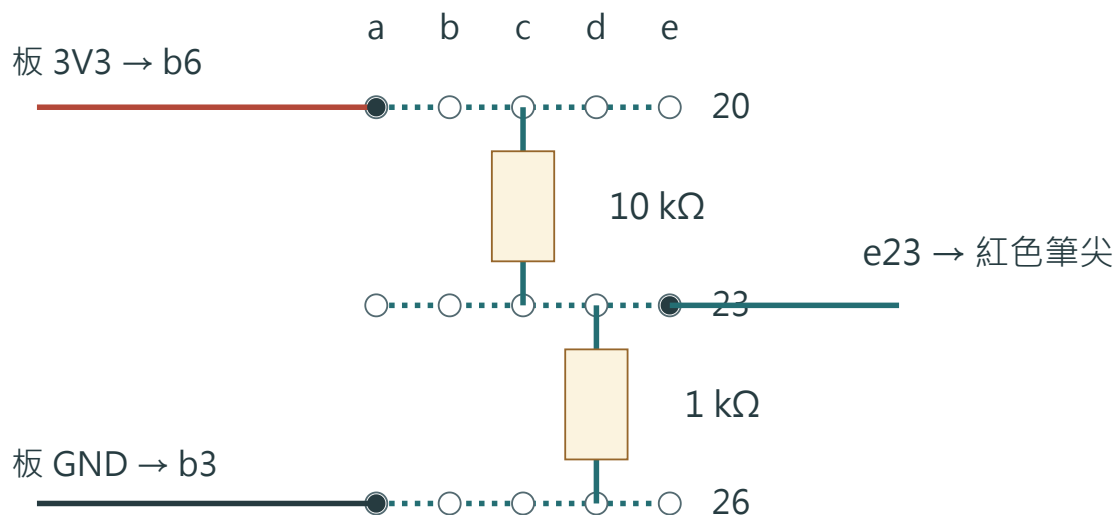
1. 每次拿一顆，與其他電路完全分離。
2. 黑色表筆插頭插電表 COM 插孔，紅色插頭插 V Ω mA 插孔；電表旋鈕選電阻區的 20k Ω 檔。兩支筆尖各碰電阻一端，不同時用手捏兩端金屬。
3. 記錄後電表旋鈕轉到 OFF。電阻不分正負，但接到電路的上下位置等一下會影響輸出。

包裝標示	完整畫面	換算後的阻值
1 k Ω	___	___ k Ω = ___ Ω
10 k Ω	___	___ k Ω = ___ Ω

1 k Ω = 1000 Ω 。理想 1 k Ω 在 20k 檔約顯示 1.00；理想 10 k Ω 約顯示 10.00。這是判讀示例，不是要求你的數值完全一樣。

先把 10 kΩ 放上面

上、下是電路的位置：靠 3V3 是上，靠 GND 是下。



黑色筆尖碰麵包板 e3 孔延長端；電阻跨不同列。

元件 / 線	兩端的麵包板孔位
供電延長線	b6 → a20
10 kΩ	c20 → c23
1 kΩ	d23 → d26
接地延長線	a26 → b3
紅筆用公對公延長線	e23 → 自由端
黑筆延長線	e3 → 自由端

第 23 列讓兩顆電阻接在一起。電阻的兩隻腳不能插進同一五孔組；上下端不能另加導線跨過電阻。輕彎腳配合孔距，不硬拉元件本體。

依表核對，確認光敏模組、開發板 GPIO4 與 GPIO5 腳都沒有接線。麵包板第 23 列相對開發板 GND 腳，會更接近 0 V 還是 3.3 V？寫下預測 ____ V，再依量測步驟比較。

量到之後，再算一次

這裡只量中間電壓，不使用 ADC，也不量電流。

確認接法後接上開發板 USB 線。電表旋鈕選直流電壓 20 V 檔；黑色筆尖碰麵包板 e3 孔延長線金屬端，紅色筆尖碰 e23 孔延長線金屬端。我的結果：_____ V。保存接線照片後移開表筆。

為什麼預期約 0.3 V？ 電流先經上面 10 kΩ，再經下面 1 kΩ，最後回到 GND。兩顆串聯，流過它們的電流相同；比較大的電阻分到比較大的電壓差。

供電電壓是 3V3 對 GND；中間電壓是第 23 列對 GND；上方電阻靠近 3V3，下方電阻靠近 GND。電壓單位 V，兩個電阻須使用相同單位。

$$\begin{aligned}\text{中間電壓} &= \text{供電電壓} \times \text{下方電阻} \div (\text{上方電阻} + \text{下方電阻}) \\ &= 3.3 \text{ V} \times 1 \text{ k}\Omega \div (10 \text{ k}\Omega + 1 \text{ k}\Omega) \\ &= 0.3 \text{ V}\end{aligned}$$

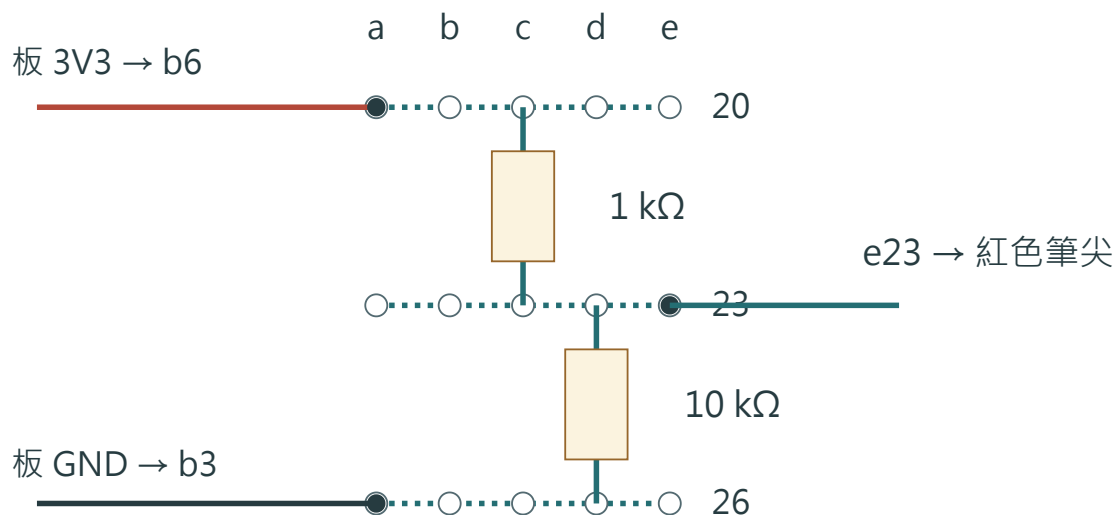
位置	理想電位	該電阻兩端差值
供電端 → 中間	3.3 V → 0.3 V	上方電阻：3.0 V
中間 → GND	0.3 V → 0 V	下方電阻：0.3 V

電表量的是中間到 GND，所以對應**下方電阻的電壓差**。若實際供電不是 3.3 V，用自己量的供電電壓與阻值重算，不把差異直接當成錯接。

交換電阻前，先拔 USB、確認開發板 PWR 電源燈熄滅、電表旋鈕轉到 OFF。

交換位置，總電阻明明沒變？

原線不動，只交換兩顆電阻的上下位置。



黑色筆尖碰麵包板 e3 孔延長端；電阻跨不同列。

開發板 USB 線保持拔除：將 1 kΩ 電阻兩腳插進麵包板 c20、c23 孔，將 10 kΩ 電阻兩腳插進 d23、d26 孔。其餘線沿用前頁。先預測中間電壓：_____ V。

核對後接上開發板 USB 線，電表旋鈕選直流電壓 20 V 檔。黑色筆尖碰麵包板 e3 孔延長線金屬端，紅色筆尖碰 e23 孔延長線金屬端。我的結果：_____ V。保存照片，再移開表筆、拔 USB 線並關閉電表。

$$\begin{aligned}\text{中間電壓} &= 3.3 \text{ V} \times 10 \text{ k}\Omega \div (1 \text{ k}\Omega + 10 \text{ k}\Omega) \\ &= 3.0 \text{ V}\end{aligned}$$

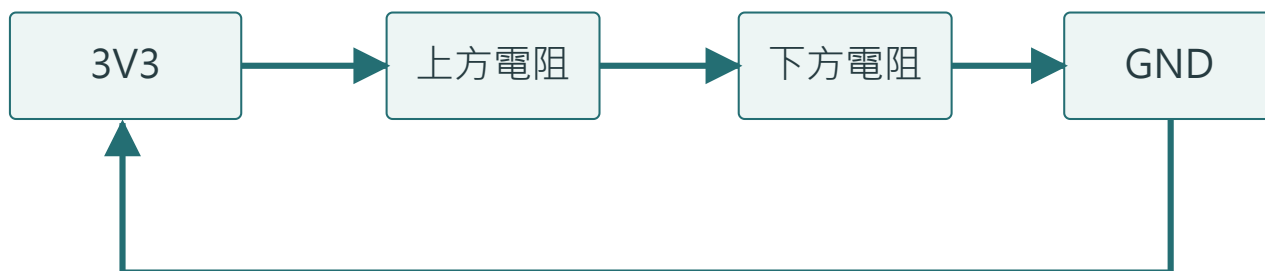
原來 10 kΩ 不是「固定扣掉幾伏特」。兩顆總阻值仍是 11 kΩ；中間到 GND 現在跨的是 10 kΩ，所以中間電壓不同。

回看 光敏模組：上方固定電阻不變，下方光敏電阻隨光改變，中間 S 的電壓就會跟著變。另一種內部接法可能方向相反，不能只憑商品名稱判方向。

兩顆電阻串聯，流過相同電流

串聯就是元件首尾相接、沒有分支；電流依序經過兩顆電阻回到電源。

兩顆串聯電阻流過相同電流



經板上電源回路返回；畫的是傳統電流方向。

公式中的 I 是電流，單位 A (安培)； V 是電壓差，單位 V (伏特)； R 是電阻，單位 Ω (歐姆)。歐姆定律 $V = I \times R$ 表示電阻兩端的電壓差等於流過它的電流乘以阻值；改寫為 $I = V \div R$ 就能算電流。 1 mA (毫安培) = 0.001 A 。

$$\begin{aligned} I &= 3.3\text{ V} \div (10000\ \Omega + 1000\ \Omega) \\ &= 0.0003\text{ A} = 0.3\text{ mA} \end{aligned}$$

這是理想計算，不是電表實測電流。交換上下位置，總阻值不變，所以理想總電流仍相同。

紙上比較，不實際新增接線	判斷
3V3 → 兩顆串聯電阻 → GND	有控制電流的阻值，是本次正常電路
3V3 → 普通導線 → GND	電源被低阻路徑直接接通，這就是短路
路徑中拔掉一端	斷路，這條路不再形成完整回路

短路**可能**造成發熱、保護動作或損壞；不代表每次一定燒壞。沒有外加電阻也不等於 $R=1\ \Omega$ ，導線電阻可能遠小於 $1\ \Omega$ ，實際電流還受電源與接觸限制。

分壓器少一顆電阻，接點電壓呢？

電路連接示意；僅作圖上推演，保持現有實物接線不動。

正常的連接 / 處理



只改標記的地方



線表示電路連接；不是實物腳序或完整接線圖。

想一想：只拆掉圖中下方的 $1\text{ k}\Omega$ 電阻，留下斷路，不用線補上。紅色筆尖仍碰中間接點，黑色筆尖仍碰開發板 GND 腳的延長線；電表讀值會接近 0 V ，還是 3.3 V ？

原理提示：高阻抗量測下，上方電阻電流變得很小，壓降接近 0 ；中間接點反而接近 3.3 V 。

只有標記處改變，其餘供電、程式與環境條件沿用正常情境。不可把未知結果直接寫成 0 、LOW 或「一定停止」。

分壓計算與電表接法

第 3 題：量兩顆電阻連接處與 GND 之間的電壓。

1. 電流方向：由電源正端 3V3 流經 1 kΩ、10 kΩ 到 GND，再經板上電源內部返回正端；兩電阻串聯流過相同電流。這不是資訊傳到 ADC 的箭頭。

2. 先算電流，再用 $V = I \times R$ 算電壓差。 I 是電流 (A)， R 是電阻 (Ω)， V 是所量兩點的電壓差 (V)。以下忽略電表取用的微小電流。

兩顆串聯，總電阻為 $1000 + 10000 = 11000 \Omega$ ；合起來兩端相差 3.3 V，所以流過每顆的電流相同：

$$\begin{aligned} I &= 3.3 \div 11000 = 0.0003 \text{ A (0.3 mA)} \\ \text{上方 } 1 \text{ k}\Omega \text{ 兩端: } V &= 0.0003 \times 1000 = 0.3 \text{ V} \\ \text{下方 } 10 \text{ k}\Omega \text{ 兩端: } V &= 0.0003 \times 10000 = 3.0 \text{ V} \end{aligned}$$

下方電阻的一端是 GND (0 V)，另一端就是中間接點，因此中間接點比 GND 高 3.0 V。從供電端往下看是 $3.3 \text{ V} \rightarrow 3.0 \text{ V} \rightarrow 0 \text{ V}$ ；兩段電壓差 $0.3 + 3.0 = 3.3 \text{ V}$ 。

把「先除以總電阻算電流，再乘以下方電阻」合併，就得到分壓公式。最後一行上下電阻都用 kΩ，比例不變：

$$\begin{aligned} \text{中間對 GND 的電壓} &= 3.3 \div (1000 + 10000) \times 10000 \\ &= 3.3 \times 10 \div (1 + 10) = 3.0 \text{ V} \end{aligned}$$

3. 電表設定與測點	答案
黑色表筆的插頭插哪裡？	萬用電表上標示 COM 的插孔
紅色表筆的插頭插哪裡？	萬用電表上標示 VΩmA 的插孔
萬用電表旋鈕選哪一檔？	直流電壓，20 V 量測範圍
黑色筆尖碰哪裡？	開發板 GND 腳；或與它相連的導線金屬端
紅色筆尖碰哪裡？	兩顆電阻的連接處；或與它相連的導線金屬端

接線時拔 USB；確認接法與檔位後，才供電量電壓。考卷未指定麵包板孔位，答出正確電路接點即可。3.0 V 是計算結果，不是本次電表實測。

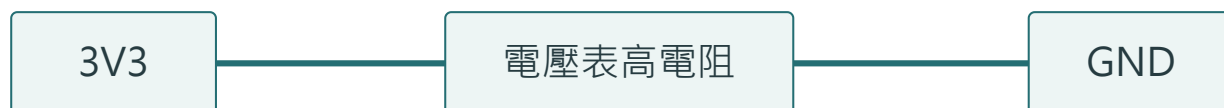
為什麼選了電壓檔，仍可能短路？

第 4 題：紅色表筆插頭接錯萬用電表插孔。

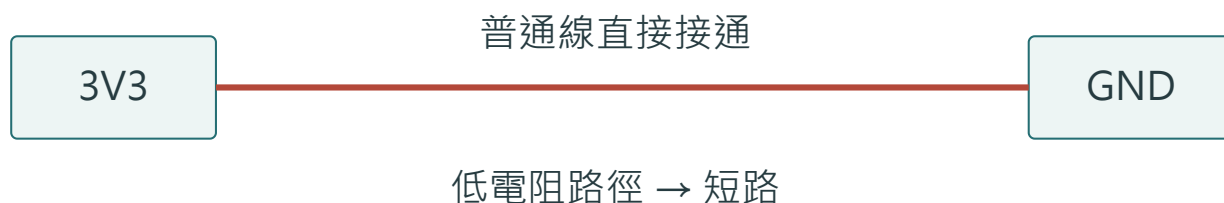
電表上標示 10A 的插孔用於電流量測。紅色表筆插在這裡時，可能經電表內部的低電阻路徑，讓電流從開發板 3V3 腳 → 紅色筆尖與導線 → 電表 10A 插孔 → 電表內部 → 電表 COM 插孔 → 黑色導線與筆尖 → 開發板 GND 腳流過，形成短路。

只將旋鈕轉到直流電壓 20 V 檔，不能代替插孔核對。保持開發板斷電，把紅色表筆的插頭改插電表 VΩmA 插孔，黑色插頭留在 COM 插孔，再確認旋鈕與測點。10A 是插孔標字，不表示短路時電流一定是 10 安培；不能通電試錯。

正常量測：



不要實作：



圖中的正常量測，是電表使用電壓插孔與電壓檔，讓很小的電流流過內部高電阻；錯誤接法可能跨出低電阻通路。兩種接法外觀看似都碰 3V3 與 GND，必須同時核對插孔、旋鈕與筆尖。表筆位置與實物照片見 p19，正常量電壓見 p21。

按一下，才記下一筆光線

用按鈕決定何時保存感測值。

固定光敏模組的位置，比較紙盒打開、蓋上時的讀值。這裡記錄光線變化，不宣稱能自動辨認所有盒子。

零件或工具	這次的作用
按鈕接 GPIO5	觸發一次記錄
光敏 S 接 GPIO4	提供當下的原始讀值
Serial Monitor	顯示序號、raw 與開機時間

預期結果：按一下印一筆；持續按住不重複印；放開再按，才新增下一筆。這需要程式記住按鈕是否已放開，不能只用「讀到 LOW 就記錄」。

Week2 的按鈕接線與 HIGH / LOW 概念沿用；本例另外加入按壓辨識，做法在本冊說明，完整程式另放 programs 資料夾。

GPIO5 這次只讀按鈕

本週前面 GPIO5 曾經輸出電壓；不能直接沿用那支程式接按鈕。

1. 拔 USB、拆掉全部外接線；收起分壓電阻與 GPIO5 電壓量測線。
2. 開啟 `programs/button_light_capture/button_light_capture.ino`。依下表填入已確認設定，Ctrl+S 儲存；依第 23 頁設定，只經開發板上標示 COM 的 USB 接頭上傳。
3. Monitor 的通訊速率選 115200 baud，應看到 `event=ready release_button_first`。關閉 Monitor、拔 USB，才依表重接。不要接 5Vin。

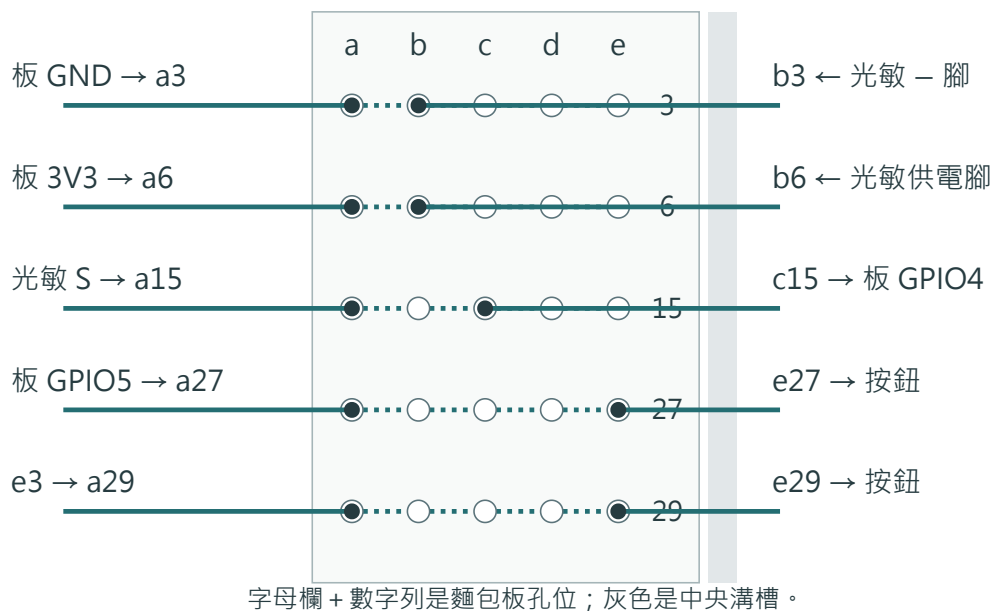
設定	圖示板型已確認後的接法
<code>PIN_LIGHT = 4</code>	光敏模組 的 S → a15；c15 → GPIO4
<code>PIN_BUTTON = 5</code>	GPIO5 → a27；a29 → e3
<code>PROFILE_CONFIRMED = true</code>	只在板型、腳序與 3.3 V 訊號已確認後設定
電源保持分列	板 GND → a3，光敏模組 — → b3；板 3V3 → a6，光敏模組 電源 → b6

按鈕四腳插麵包板 **e27、f27、e29、f29** 孔，跨中央溝槽。開發板斷電時，在 b27、b29 孔各插一條延長導線；電表旋鈕選通斷檔，兩支筆尖各碰一條線的金屬端。放開按鈕不通，按下才通。不符先調整方向，不通電。a27 與 e27 相通，a29 與 e29 相通，不把杜邦母頭套在按鈕短腳。

原程式的腳位設定為 -1、確認值為 false，表示尚未確認接法。圖示的 GPIO4、GPIO5 僅供已核對板型使用。INPUT_PULLUP 是程式啟用晶片內上拉電阻的輸入設定，不是把 3V3 電源直接接按鈕。

按鈕與光敏各接一個 GPIO

按鈕與光敏共用 GND，但訊號線分開。



- 按鈕四腳插 e27、f27、e29、f29，跨過麵包板中央溝槽。
- a27 與 e27 相通；a29 與 e29 相通；a15 與 c15 相通。
- 光敏模組 中間供電腳接 b6；不是 GPIO5，也不是 5Vin。

接 USB 前核對：第 6 列是 3V3；第 3、29 列是 GND；第 15 列是光敏訊號；第 27 列是按鈕訊號。第 15、27 列不能互接。

打開、蓋上，各按一次

保持感測器位置不動，只有遮光條件改變。

連接開發板上標示 COM 的 USB 接頭，Monitor 的通訊速率選 115200 baud。先放開按鈕至少 0.1 秒，再按住約半秒。

操作	預期結果
室內光，按一次	sample=1，一筆 raw
按住兩秒	不新增資料
放開，遮光，再按一次	sample=2，一筆新的 raw
按開發板的 RST 按鈕	重新顯示 ready，記錄序號重新開始
開機時壓住按鈕	放開後重新按，才記錄第一筆

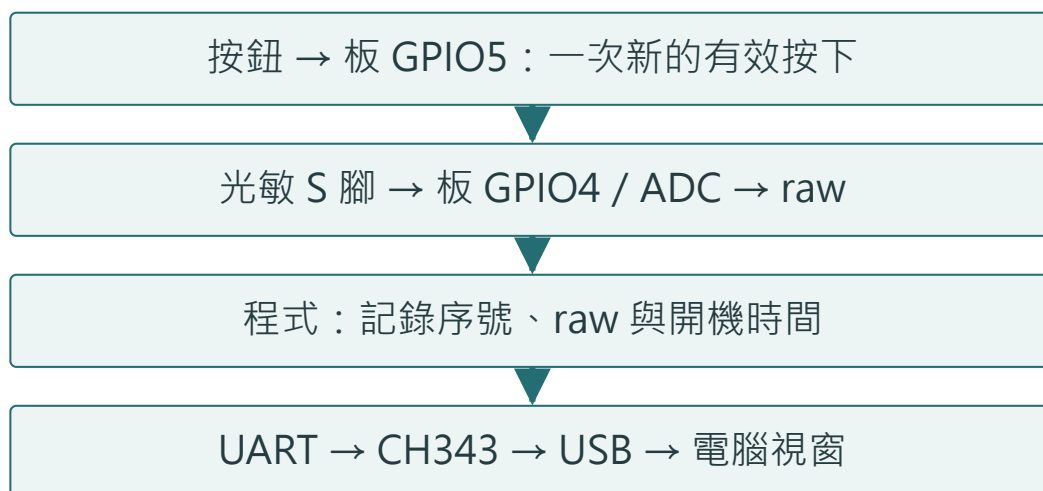
預期輸出格式；raw 與時間是示例，不是實測：

```
event=sample sample=1 raw=420 endpoint=0 uptime_ms=3040
event=sample sample=2 raw=1020 endpoint=0 uptime_ms=6040
```

sample 是記錄序號；uptime_ms 是開機後毫秒；endpoint=1 表示 raw 是 0 或 4095，不直接證明斷線。依第 34 頁將文字另存成檔案。

視窗沒有 ready 訊息：查 Arduino IDE 的 Port 連接埠、Serial Monitor 的 115200 baud 通訊速率，以及程式是否上傳成功。出現 check_configuration：斷電核對板型、兩個不同接腳與程式中的 PROFILE_CONFIRMED 設定。

按鈕決定何時，感測器決定數字



digitalRead 讀 GPIO5 的 HIGH / LOW，判斷按鈕。**analogRead** 讀 GPIO4 的原始數字，取得光敏訊號。它們讀的是不同腳位、不同內容。

程式部分	對應剛完成的操作
<code>pressed(now)</code>	只在一次新的有效按下回傳 true
<code>analogRead(PIN_LIGHT)</code>	在那次按下時取一筆 raw
<code>++sampleNumber</code>	記錄序號加一
<code>Serial.printf</code>	把序號、raw 與時間送到 Monitor

Monitor 顯示的是已送出的歷史文字，不會因光線繼續改變而改寫上一行。要看新資料，放開再按一次。

兩條電流路徑，兩個訊號接點

按鈕按下時：傳統電流方向



光敏電路：供電與感測接點



按鈕按下：GPIO5 經按鈕連到 GND，電壓接近 0 V，讀到 LOW；晶片內的上拉電阻限制這條路徑的電流。

按鈕放開：按鈕這一段不通，GPIO5 由晶片內上拉電阻連著 3.3 V，電壓接近 3.3 V，讀到 HIGH。

光敏電路持續由 3V3 供電。S 是分壓的中間接點，GPIO4 讀取它相對 GND 的電壓。GPIO4 不替模組供電，也不要額外用線把 GPIO4 接到 GND。

為什麼按住不會一直印？

程式不只看 LOW，還記住是否已經放開。

按鈕狀態	程式記住的狀態	處理結果
放開且穩定	armed = true	準備接受下一次按下
按下且穩定，armed 為 true	改成 false	記錄一筆
繼續按住，armed 為 false	維持 false	不新增記錄

armed 是本程式的變數名稱，表示已準備接受新的按下。它不是另一個零件。

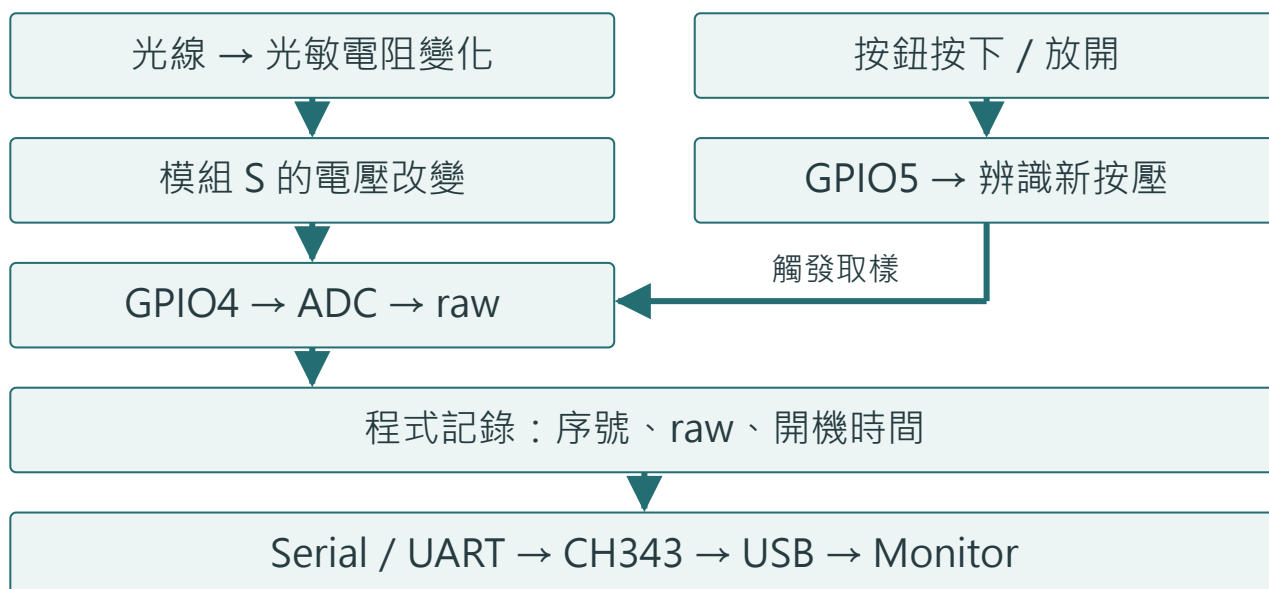
按鈕接觸的瞬間可能短暫跳動。程式記下讀值改變的時間 `changedAt`；同一讀值維持 40 ms 才接受，這種處理叫去抖動。40 ms 是本例選擇，不是所有按鈕的固定規格。

`millis()` 是開機後經過的毫秒；`now - changedAt` 是本次讀值已維持多久。loop 持續讀按鈕。

開機時 `armed = false`。即使當時按住按鈕，也須先放開、再按，才留下第一筆資料。

資訊與量測位置

Q5-1 · 資訊流



完整回答應包含：光線改變光敏模組 S 訊號腳的電壓；ESP32 晶片內的 ADC 將 GPIO4 腳電壓轉成 raw；按鈕由 GPIO5 腳輸入，新的按下觸發記錄。程式經 Serial 送出文字，由 UART 串列通訊傳到板上的 CH343 晶片，再轉成 USB 資料到電腦視窗。這些是資訊傳遞，不是光敏電流向電腦。

ADC 讀 GPIO4 相對共同 GND 的電壓。GPIO5 讀按鈕狀態；本週基本整合中，按鈕觸發保存當下的一筆光線資料，不負責供應光敏模組的電源。

Q5-2 · 只斷開 S 線

模組仍有 3V3 與 GND 供電，但 S 與 GPIO4 的訊號連接斷開。若 GPIO4 沒有其他電路固定電壓，會浮接；ADC 仍可能輸出數字，不能把它當成有效光線資料，也不能保證為 0。

斷開的是感測器到 ADC 的連接，不是 USB 或電源。有數字不代表有可信的光線資料；反例只作紙上推演。

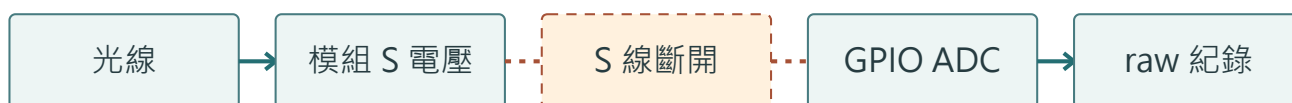
光敏模組有電，不等於 ADC 有訊號

資訊處理示意；僅作圖上推演，保持現有實物接線不動。

正常的連接 / 處理



只改標記的地方



箭頭表示資訊處理順序，不是供電或電流路徑。

想一想：保留模組的 3V3 與 GND，只在圖上斷開 S 到 GPIO 的線。raw 會一定變成 0 嗎？

原理提示：模組可能仍正常供電，但 ADC 腳失去可靠訊號；讀值不能當光線資料，也不保證是 0。

只有標記處改變，其餘供電、程式與環境條件沿用正常情境。不可把未知結果直接寫成 0、LOW 或「一定停止」。

改一處後，哪一段會改變？

分壓器少一顆電阻，接點電壓呢？

原本理想分壓為 $3.3\text{ V} \times 1\text{ k}\Omega \div (10\text{ k}\Omega + 1\text{ k}\Omega) = 0.3\text{ V}$ 。電表電壓檔的輸入電阻很大（稱輸入阻抗）；下方電阻拆掉後，若它遠大於 $10\text{ k}\Omega$ ，流過上方電阻的電流很小，接點就接近 3.3 V 。拆掉與短接不同：下方改成導線，接點近 0 V ，上方仍限流；改拆上方而留下下方，接點也近 0 V 。只在圖上推演。

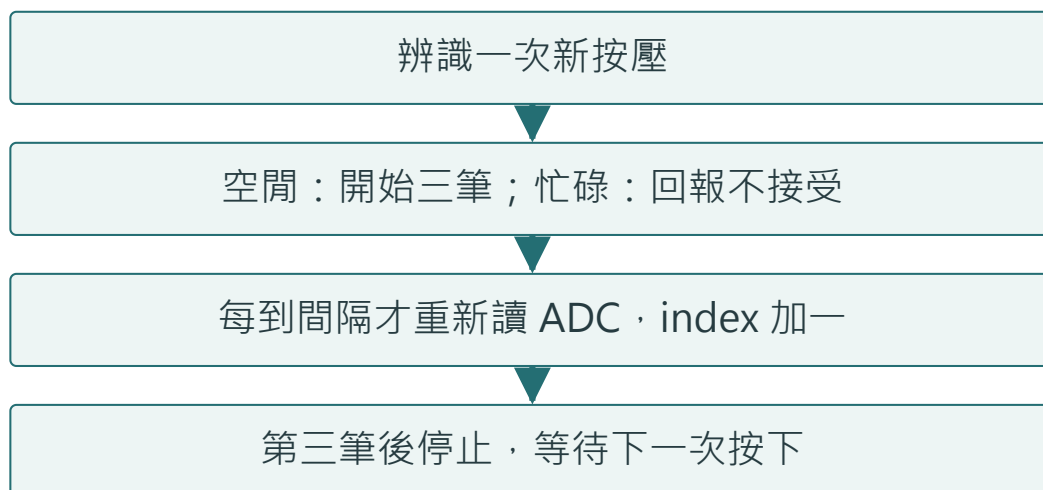
光敏模組有電，不等於 ADC 有訊號

S 接點仍可能隨光線改變，但 ADC 讀的是自身腳位。若腳位斷開且沒有其他電路固定其電壓，就稱為浮接；數值可能受干擾，不能當光線資料。供電與訊號連線分別核對；斷電後查 S 線與共地，不把 GPIO 接 5 V 製造對照。

一次按下，取得三筆新資料

對應 Main 「按一次，自動取三筆」。

人只按一下，後面兩次由程式自動安排。這次取得的三筆合稱一批；`batch` 記第幾次成功啟動，`index` 記這次的第幾筆。還在讀取時被忽略的按下，不增加 `batch`。



題目必要行為	參考程式的做法
一次有效按下啟動一批	<code>batch</code> 加一， <code>index</code> 從 1 開始
第一筆不額外等待	開始時呼叫 <code>capture</code>
後兩筆至少間隔 200 ms	<code>millis</code> 時間差達門檻才取樣
放開也要完成三筆	<code>collecting</code> 記住仍在取樣
忙碌時的新按壓不排隊	印 <code>batch_busy</code> ，不增加 <code>batch</code>
第三筆後停止	<code>collecting</code> 改為 <code>false</code>

三筆、至少 200 ms、不排隊是題目要求；時間由程式計算，不要求人精準按在某個毫秒。40 ms 去抖、英文欄位名稱及函式拆法是參考實作的選擇；題目也接受清楚的中文顯示。

建立 three_light_samples

1. 在 Arduino IDE 開啟 `programs/three_light_samples/three_light_samples.ino`。
2. 選 **檔案** → **另存新檔**，建立自己的 `three_light_samples` 工作副本，不在單筆程式尾端拼貼。
3. 填入與單筆範例相同、已確認的 `PIN_LIGHT`、`PIN_BUTTON` 與 `PROFILE_CONFIRMED`。參考檔仍預設 `-1 / false`，不能直接當作已確認接法。
4. **Ctrl+S** 儲存。移開表筆、拔 USB、拆除板上外接線，電表旋鈕轉到 **OFF**。
5. 只將開發板上標示 **COM** 的 USB 接頭連到電腦。IDE 使用本冊 p23 的圖示板設定，先按 **Verify**，再按 **Upload**。
6. 拔 USB，依本冊 p52 的按鈕 + 光敏接線圖接回；確認後接上開發板上標示 **COM** 的 USB 接頭，開 **Serial Monitor**，通訊速率選 **115200 baud**。
7. 先放開按鈕至少 0.1 秒。應看到 `event=ready release_button_first`；若是 `check_configuration`，先核對設定，不猜腳位。

這次沿用的接點	功能
GPIO4 ← c15 / 光敏模組 S	讀取光敏訊號
GPIO5 → a27 ; a29 → e3	讀按鈕 ; e27、f27、e29、f29 跨溝槽
GND → a3 ; 光敏模組 – → b3	共地
3V3 → a6 ; 光敏模組 中間 → b6	模組供電

只適用本冊已核對的板型與模組。不能讓 GPIO5 維持本週前段的 OUTPUT 程式來接按鈕。

不停住程式，也能等 200 ms

`millis()` 取得開機後毫秒。`lastSampleAt` 記住上一筆的時間；兩者相減，才知道間隔到了沒有。

有效按下：立刻取第 1 筆，記下時間
間隔不足 200 ms：繼續讀按鈕，不取第 2 筆
間隔達到 200 ms：重新讀 ADC，取第 2 筆
再等至少 200 ms：重新讀 ADC，取第 3 筆，停止

為什麼不用 `delay(200)`？`delay` 會讓這段程式暫停；期間可能無法辨識新的短按，更無法即時回報忙碌。本例讓 `loop` 持續執行，用時間差決定取樣時機。

忙碌判斷先處理新的按下。若新按下被辨識時，第三筆尚未完成，就印 `batch_busy`；不在第三筆完成後補做這次按下。

時間晚了就讀當下的一筆。例如第二筆到 230 ms 才執行，就從那筆重新等至少 200 ms，不連續補印兩筆來假裝準時。

按住不放不會再開一批。`pressed` 只有在放開後的新按下才回傳 `true`。忙碌時被忽略的按下也已被消耗，必須放開再按。

按兩次，程式各讀三筆

Reset 後先放開至少 0.1 秒。室內光下按住兩秒，放開半秒，再遮光按住兩秒。

batch=1 index=2 表示第一次啟動的第 2 筆；raw 是光敏數值。uptime_ms 是程式讀到這筆時，距離開機經過的毫秒，不是人按鈕的秒數。endpoint=1 表示 raw 為 0 或 4095，其餘為 0。以下為示例，不是實測。

```
event=sample batch=1 index=1 raw=420 endpoint=0 uptime_ms=3040
event=sample batch=1 index=2 raw=422 endpoint=0 uptime_ms=3240
event=sample batch=1 index=3 raw=419 endpoint=0 uptime_ms=3440
event=sample batch=2 index=1 raw=1020 endpoint=0 uptime_ms=6040
event=sample batch=2 index=2 raw=1025 endpoint=0 uptime_ms=6240
event=sample batch=2 index=3 raw=1018 endpoint=0 uptime_ms=6440
```

核對項目	預期
每批筆數	三筆，index 依序 1、2、3
同批相鄰時間相減	至少 200 ms
按住超過三筆完成時間	不新增第四筆
在第一筆後提早放開	第二、三筆仍會完成
放開再按	batch 加一，index 從 1 開始

A 的程式說明：capture(now) 每次都執行 analogRead(PIN_LIGHT)，把新讀值存進區域變數 raw。接受新按壓後立即呼叫一次；之後距離上一筆至少 SAMPLE_GAP_MS 時再呼叫，第三筆後停止。每次都有重新量測，光線穩定時仍可能讀到相同數字。用自己的程式指出相應的讀取敘述、執行時機與停止條件。

忙碌、重新開始與接線問題

先放慢，讓手動操作來得及。另存測試副本，把 `SAMPLE_GAP_MS` 暫改為 `1000`，表示兩筆相隔至少一秒。依 p61 的斷電、裸板上傳及接回順序換程式，不能帶電拆線。

按開發板的 RST 按鈕、放開按鈕。按住約半秒、放開約半秒、再按住；第二次按下要在第三筆出現前。預期看到：

```
event=ignored reason=batch_busy
```

這句表示「仍在讀取，這次按下不接受」。等三筆完成，確認沒有補做一次；同樣用慢速設定檢查提早放開及中途按開發板的 RST 按鈕。未測到就如實記錄，不必反覆搶按。

完成後把 `SAMPLE_GAP_MS` 恢復 `200` 並重新上傳，展示正常的兩次按下。正式間隔看程式自動印出的時間差；一秒測試只驗證按鈕行為，不能當成 `200 ms` 計時證據。不要求在 `200 ms` 模式手動重做快速按壓。

現象或操作	核對方法與預期
忙碌按下後一直壓住	第三筆後停止，不延後新增批次
取樣途中按開發板的 RST 按鈕	中止這批；放開再按，重新從 <code>batch=1</code> 開始
印出 <code>check_configuration</code>	核對確認值、腳位及兩腳不同；不先換線
<code>raw</code> 固定 <code>0 / 4095</code>	用電表量模組 S 腳對開發板 GND 腳的電壓，再斷電查線
Serial Monitor 沒文字	查開發板 USB 接頭、IDE 連接埠與視窗通訊速率，確認程式已上傳
遮光方向與預期相反	保存資料，斷電核對模組與接法，不反接供電

本冊為參考解法。主機測試、編譯不能代替實物按鈕、ADC 與學生操作驗證；本次沒有新增硬體實測。

開啟對應作品的程式

以下是參考實作；符合題目要求的其他寫法也可以。

在本週 **programs** 資料夾中找到下表的程式。每個子資料夾內都有同名 **.ino**，用 Arduino IDE 開啟後另存自己的工作副本。

考卷中的作品	程式資料夾	本冊操作起點
B · 可以暫停的遮光計數器	shade_counter	p66
C · 桌面光線觀測器	light_observer	p71
D · 按一下，留下光線快照	light_snapshot	p78

1. Arduino IDE 選「檔案 → 開啟」，選資料夾內的 **.ino**。
2. 選「檔案 → 另存新檔」，保留相同程式名稱，存進自己的作品資料夾。
3. 每次換程式，重新填入該程式的設定；前一支程式的值不會自動帶過來。
4. 程式預設 **PROFILE_CONFIRMED=false**、**GPIO** 為 **-1**。這表示尚未確認實物接法，不是可直接照填的接腳。

本冊使用 YD-ESP32-S3 開發板上標示 COM 的 USB 接頭：Arduino IDE 選 **ESP32S3 Dev Module**、本機實際 Port、**USB CDC On Boot: Disabled**；Serial Monitor 的通訊速率選 **115200 baud**。Flash / PSRAM 等其餘設定見本冊 p23，每次換程式重新核對。

改線前拔 USB 與其他電源。換程式時先拆外接線，只接裸板 USB 上傳；上傳成功後再拔 USB 接線。GPIO5 不能帶著輸出 HIGH 的程式直接接按鈕到 GND。

先讓按鈕切換啟動與暫停

材料：ESP32-S3、光敏模組、一顆按鈕、麵包板、杜邦線。不接 OLED；電腦會顯示 `mode`（啟動或暫停）及 `count`（遮光次數）。

開啟 `programs/shade_counter/shade_counter.ino`，在程式開頭搜尋並填入：

設定	填入什麼
<code>PIN_LIGHT</code> 、 <code>PIN_BUTTON</code>	開發板連光敏 / 按鈕的 GPIO 編號；圖示分別填 4、5
<code>INDOOR_MIN</code> 、 <code>INDOOR_MAX</code>	本組同位置、未遮光十筆的最小與最大 raw
<code>SHADE_MIN</code> 、 <code>SHADE_MAX</code>	同輪遮光十筆的最小與最大 raw
<code>PROFILE_CONFIRMED</code>	電源、模組腳序及 GPIO 接法確認後填 <code>true</code>

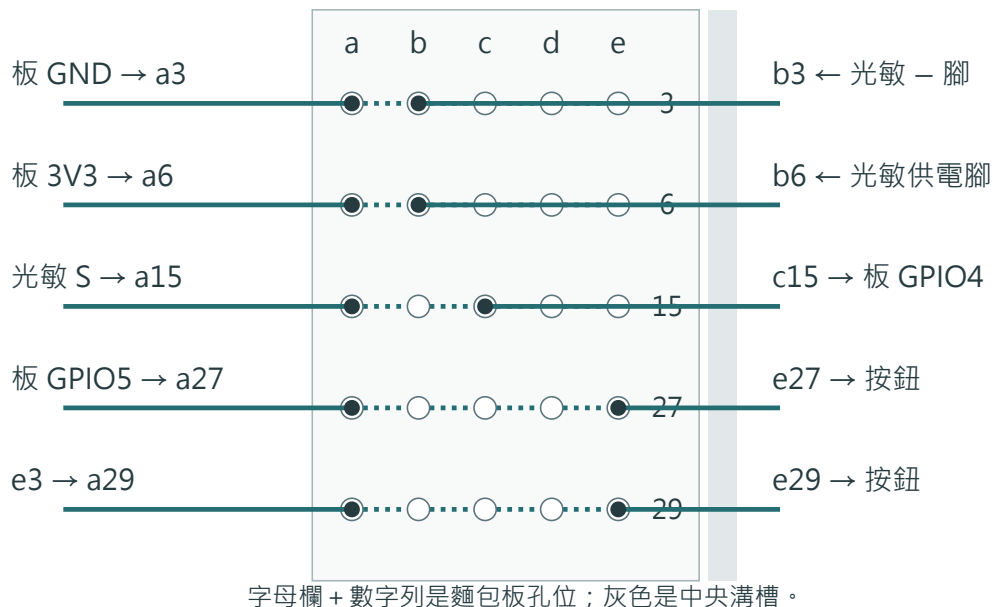
若沒有基準資料，先完成本冊 p35「室內光十筆，遮光十筆」：固定位置、各保存十筆，從原始資料找最大與最小值。不用示例數字代替自己的量測。

1. 接線保持拆除，只用開發板上標示 COM 的 USB 接頭連到電腦。
2. Ctrl+S 儲存，按勾號 **Verify** 檢查編譯，再按箭頭 **Upload**。
3. 上傳完成，拔 USB，依下一頁接光敏與按鈕。

兩段資料重疊、碰到 0 / 4095，或尚未填好設定，程式會印 `blocked` 而不計數。先查資料與設定，不靠反接電源改變數字方向。

光敏與按鈕各用一條訊號線

只適用已核對腳序與 3.3 V 電氣條件的課堂模組。



板 GND 接 a3、板 3V3 接 a6。光敏 - 接 b3、中間電源腳接 b6、S 接 a15；c15 接 GPIO4。按鈕跨中央槽插 e27、f27、e29、f29；GPIO5 接 a27，a29 接 e3。

開發板斷電時，用電表通斷檔，兩支筆尖分別碰麵包板 b27、b29 孔的延長導線金屬端。放開按鈕不通，按下才通。核對後關閉電表、移開表筆，再接開發板 USB 線；第 3、6 列及第 15、27 列不得互接。

開 Monitor 的通訊速率選 115200 baud，先放開按鈕。預期先出現 mode=PAUSED count=0。若不是此訊息，查正在上傳的檔名與 Port，不在通電中移線。

暫停不歸零，恢復不補算

每次遮住或移開至少維持一秒；按鈕按一下後放開，按下與放開各約半秒。依序做，不把中間操作省略。

操作	預期 Monitor 新增或保留的狀態
未遮住，放開按鈕，按開發板的 RST 按鈕	PAUSED · count=0
按一下，再放開	RUNNING · count=0
遮住並保持三秒	只增加到 count=1
移開一秒，再遮住	count=2
保持遮住，按一下暫停	PAUSED · count=2
暫停中移開、遮住兩次	count=2 · 不新增計數
遮住時按一下恢復	RUNNING · 仍為 2
移開一秒，再遮住	count=3
按住按鈕兩秒	只切換一次至 PAUSED · 仍為 3

放開再按開發板的 RST 按鈕，回到 PAUSED、0。另試開機前先遮住：沒有真正發生「未遮住 → 遮住」，不應自行加一。按住按鈕開機，也須先放開再按才接受操作。

用這段 Monitor 畫面展示計數與暫停；Main 練習 B 寫本組採用的判斷規則與數字依據。raw 方向由本組資料決定，不固定要求「遮光一定比較大」。

數的是「變成遮住」，不是「仍然遮住」

程式用 `running` 記住啟動或暫停；`wasShaded` 記住上一筆是否遮住；`known` 表示已經有可比較的上一筆。這三個都是程式中的記憶，不是新零件。

按鈕 → GPIO5 → 一次新按壓 → 切換 `running`
 光線 → S 電壓 → GPIO4 → `raw` → 比較分界
 上一筆未遮住 + 這一筆遮住 + 正在計數 → `count` 加一
`mode`、`count` → Serial → CH343 / USB → Monitor

每 50 ms 讀一次光敏。暫停時仍更新光線狀態，但不加次數；切換模式後，第一筆只用來建立比較起點。因此恢復時已經遮住，也不補算一次。

只改程式的一處，在紙上推演	會發生什麼
只要 <code>shaded</code> 就加一	持續遮住，每次讀取都可能再加一
暫停時完全不更新上一筆	恢復時可能把暫停期間的變化誤算成新事件
模式切換後不重新建立起點	同時恢復與遮光時，可能把恢復動作誤算進去

電流路徑仍是本冊 p70 的兩條回路；暫停只是程式不加次數，不是切斷感測器供電。`raw=0 / 4095` 時，本範例暫不分類；下一筆正常資料重新建立比較起點。

電源持續供電，按鈕只送操作訊號

按鈕按下時：傳統電流方向



光敏電路：供電與感測接點

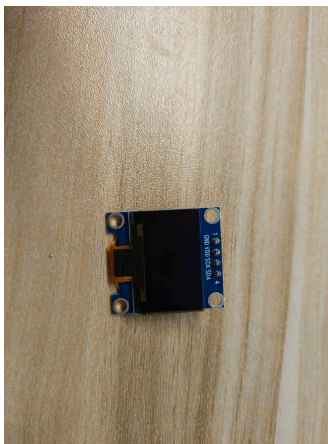


按鈕按下：3.3 V 經晶片內上拉電阻、按鈕回到 GND；GPIO5 讀到接近 0 V。光敏由另一條 3V3 → 模組內電阻 → GND 的路徑工作，GPIO4 只讀 S 對 GND 的電壓。

紙上反例：若把 GPIO5 當光敏電源，按下可能改變供電而不只是送操作訊號。若只拔 S 線，模組仍有電，卻不能再把 GPIO4 的數字當作可靠的光線資料。不要在通電中拔線測這些反例。

辨認四條 OLED 線

OLED 是用小發光點顯示文字的螢幕。這兩個作品使用四針、128×64 的 I2C 模組；128×64 是橫、縱像素數。I2C 是用資料線與時脈線傳送資訊的通訊方式，另需兩條電源線。



已購 OLED 的辨識照片。依手上模組絲印接線，不靠綠色或照片左右位置。

這片模組的標字	接法與用途
GND 接地腳	接開發板 GND，作共同參考與電源回路
VDD 供電腳	本課已確認的模組接 3V3；其他模組可能標 VCC
SCK 時脈腳	接 GPIO9；這片 I2C OLED 的 SCK 就是程式所稱的 SCL
SDA 資料腳	接 GPIO8，傳送畫面資料

找不到 SCL 標字也能接：本片印的是 SCK。ESP32 上找印著 8、9 的接腳，分別接 SDA、SCK；不是從旁邊數第 8、9 根針。程式以 `PIN_SDA = 8`、`PIN_SCL = 9` 指定用途。

教師先前測過通訊位址 0x3C 與 SSD1315 相容驅動。位址是裝置的通訊編號，驅動是控制螢幕的方式；不能只憑四個腳位標字推定型號，下一頁先掃描確認位址。

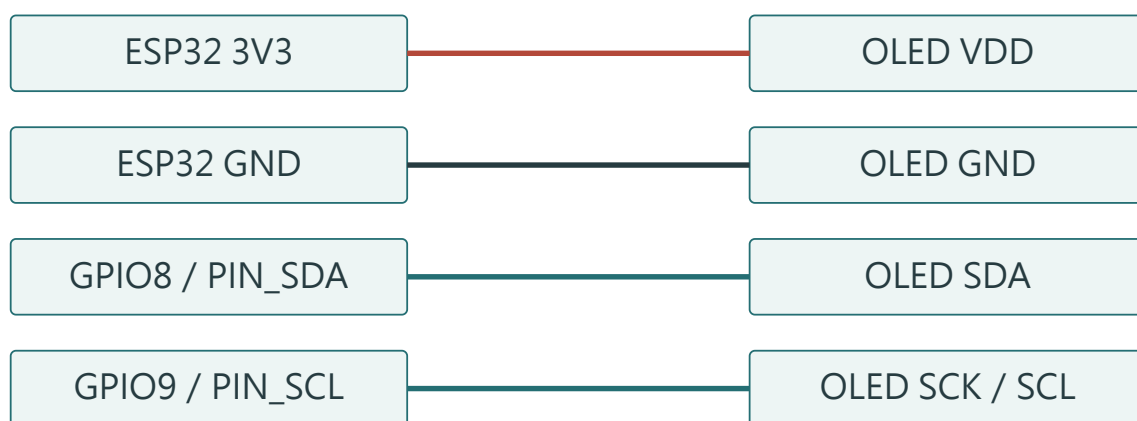
開發板 USB 線保持拔除。先核對本片腳序、3.3 V 供電與訊號相容性；控制器、通訊方式或解析度不同時，不硬套程式。不能因螢幕沒亮就提高供電電壓。

先只接 OLED，確認有回應

位址是通訊時用來選擇裝置的編號。0x3c 是十六進位寫法，不是 GPIO 編號，也不是電壓。

1. 拔 USB，拆下光敏和按鈕線；只留裸板。
2. Arduino IDE 開啟 `programs/week05_i2c_check/week05_i2c_check.ino`；檔案已放在本週 `programs` 內。
3. 程式的 `PIN_SDA`、`PIN_SCL` 填已確認的開發板 GPIO 編號；圖示分別為 8、9。核對供電與訊號電壓後，才將 `ELECTRICAL_PROFILE_CONFIRMED` 改為 `true`。
4. 只連接開發板上標示 COM 的 USB 接頭，Verify、Upload。成功後拔 USB，再按下圖連 OLED。

連接關係示意；依模組絲印辨認，不是排針實際順序



GPIO8、GPIO9：開發板接腳
PIN_SDA、PIN_SCL：程式設定名稱

供電可用麵包板分接：板 GND → a3、OLED GND → b3；板 3V3 → a6、OLED VDD → b6。同列 a~e 五孔相通，不跨中央槽。SDA 接 GPIO8，SCK 接 GPIO9。孔位可另選，但 GND 與 3V3 必須分列，不能相接。核對後插 USB，Monitor 的通訊速率選 115200 baud。

有回應後，準備顯示文字

掃描每次開機只跑一次，沒看到文字時按開發板的 RST 按鈕 重看。以下為格式示例，不是本次實測：

```
event_type=i2c_ack address_7bit=0x3C model=unknown  
event_type=scan_finished found=1 model_verified=false
```

ACK 是對方的確認回應。若 found=0，拔 USB 查供電、共同 GND、SDA、SCK。若有多個或不符模組資料的位址，先核對，不任選一個試。只有位址回應，還不能證明控制器型號或顯示正常。

1. Arduino IDE 開左側「程式庫管理員」，搜尋 **U8g2**，作者 **Oliver Kraus**，安裝 **2.36.15**。
2. Wire 隨 ESP32 套件提供，不另安裝同名程式庫。
3. 記下程式 PIN_SDA、PIN_SCL 的 GPIO 編號、掃描顯示的 7-bit 通訊位址，以及適用的螢幕驅動。7-bit 表示位址用七個二進位位元表示；接下來兩支程式要填同一組已確認設定。

U8g2 負責把文字畫成像素，再送到螢幕；它不會替我們判斷光線或保存最大值。

若編譯時找不到 U8g2lib.h，先確認程式庫安裝；若有 ACK 卻畫面錯位，核對控制器與解析度。這兩種問題不能靠加高電壓處理。

把目前值和最大、最小值放上螢幕

開啟 `programs/light_observer/light_observer.ino`。本作品不接按鈕；每 200 ms 取一筆新資料，再同時更新 OLED 與 Monitor。

搜尋設定	填入內容
PIN_LIGHT	光敏模組 S 腳所接的開發板 GPIO 編號；圖示填 4
PIN_SDA、PIN_SCL	開發板連 OLED 的兩個 GPIO 編號；圖示分別填 8、9
OLED_ADDRESS	掃描並核對後的 7-bit 位址，如 0x3C，不乘 2
#define OLED_CONTROLLER	本片適用的 1306 或 1315
PROFILE_CONFIRMED	確認電氣條件與所有接點後填 true

1. Ctrl+S，拔 USB、拆掉外接線。
2. 只連接開發板上標示 COM 的 USB 接頭，Verify、Upload；成功後拔 USB。
3. 依下一頁接回 OLED，新增光敏。GPIO4、8、9 各做不同的事，不能互接或重複指定。

程式初始將 MIN / MAX 當作「尚無資料」。取得第一筆後才有真正的最小與最大值，不能因為變數初始寫 0 就宣稱量過 0。

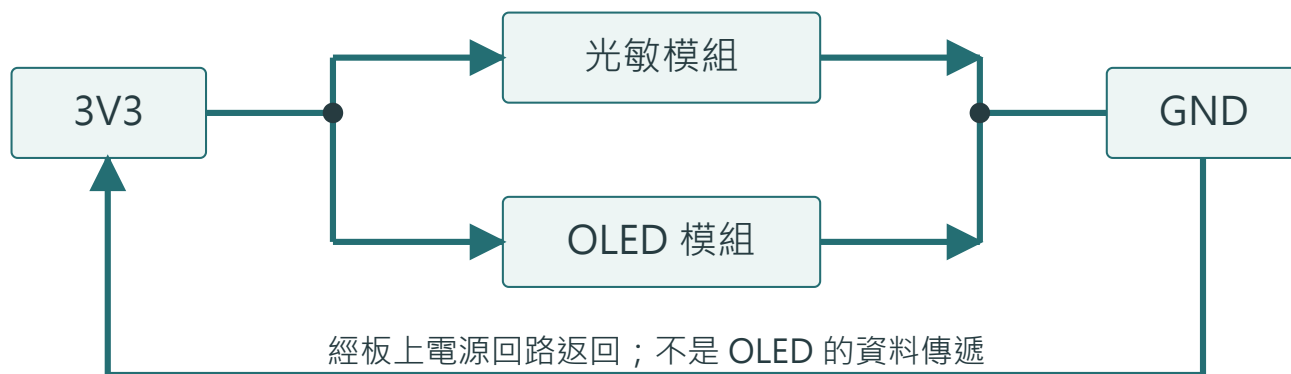
共用電源，分開訊號

USB 拔除；拆掉按鈕及 GPIO5 線，保留光敏與 OLED。

開發板 / 模組接腳	對應的麵包板孔位或開發板接腳
板 GND、3V3	a3、a6
OLED GND、VDD	b3、b6
光敏 -、中間電源腳	c3、c6
光敏 S	a15；c15 接 GPIO4
OLED SDA、SCK	開發板上已確認的資料 / 時脈腳；圖示為 GPIO8、GPIO9

a3 ~ e3 內部相通，a6 ~ e6 內部相通；兩列彼此不通。不要把光敏 S 插在電源列。核對完連接開發板上標示 COM 的 USB 接頭，Monitor 的通訊速率選 115200 baud。

供電回路示意：傳統電流分成兩條支路



兩個模組是並聯供電，不是感測電流先經光敏、再流到 OLED。訊號另走：光敏 S → GPIO4；ESP32 的 SDA / SCL GPIO 與 OLED 通訊。上圖不是排針順序。

若出現 blocked，依訊息查設定或 OLED 回應。通電後不動線；要改接先拔 USB。

RAW 會變，MIN 和 MAX 留住範圍

上電可能短暫顯示 WAIT；第一筆出現後，三行都有數字。下面用指定數字示範規則，實物應顯示自己的讀值。

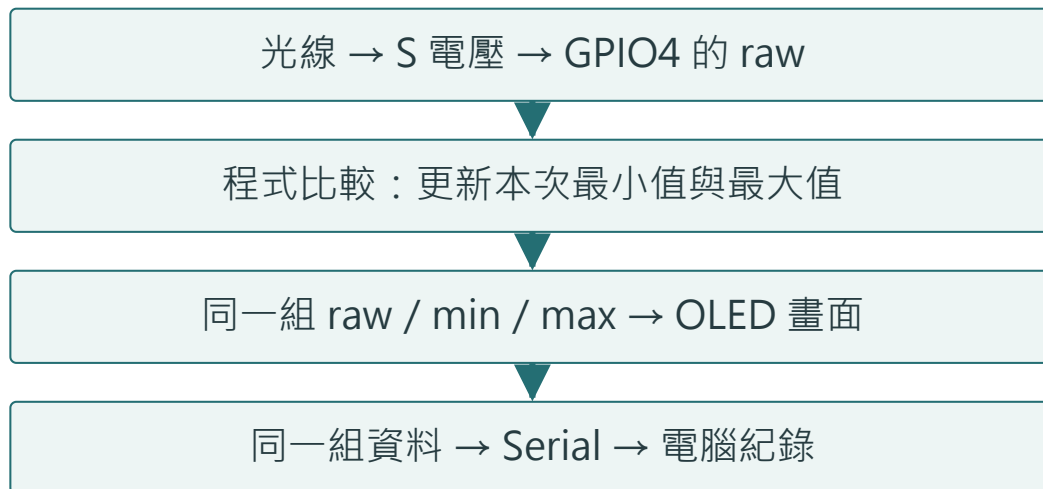
依序取得的資料	OLED 三行	Monitor
第一筆 480	RAW 480 / MIN 480 / MAX 480	raw=480 min=480 max=480
再取得 980	RAW 980 / MIN 480 / MAX 980	raw=980 min=480 max=980
再取得 510	RAW 510 / MIN 480 / MAX 980	raw=510 min=480 max=980

實際操作：保持光線一秒、遮住一秒、移開一秒。確認畫面每秒至少更新一次， $\text{MIN} \leq \text{RAW} \leq \text{MAX}$ ；光線回復時，MIN / MAX 不應直接跟著回到目前值。

按開發板的 RST 按鈕後，原範圍清除。重啟後第一筆的 RAW、MIN、MAX 相同，不沿用前一輪最大值。OLED 與 Monitor 的一致性用畫面展示。

這支程式每筆都保留，包含 0 / 4095；極端數字也會影響範圍。持續端點值須查電路及量測範圍，不能把它解讀為精確照度。

一次讀值，同時供應兩個畫面



C 的程式說明：minimum 記最小值，maximum 記最大值。第一筆把兩者都設成 raw；之後只有更小才改 minimum、更大才改 maximum。完整程式在 `programs/light_observer/light_observer.ino`；回答時指出自己的第一筆處理及後續比較敘述。

一次 ADC 讀值用在同一輪的比較、畫面與紀錄，不在每個輸出前重讀一次。

例如恢復後 RAW=510、MIN=480、MAX=980，表示本次曾量到比現在更小及更大的讀值。這是示例；只有範圍不能還原變化順序或持續多久。若三者相同，表示本次尚未記到不同值。

U8g2 的 buffer 是暫放下一張圖的記憶體；clearBuffer 清掉舊圖，drawStr 在指定位置畫字，sendBuffer 才傳到 OLED。它們不會自動更新 MIN / MAX。

紙上反例：若每次都把 minimum=maximum=raw，螢幕仍有三行，卻完全沒有歷史範圍。若刪掉 sendBuffer，Monitor 仍可能更新，但螢幕不會收到新畫面。

供電路徑見 p75。只拔 SDA 不會切斷 OLED 的電源，卻會破壞通訊；有電不等於有新文字。這些反例不在通電實物上操作。

加回按鈕，保留按下那一刻的數字

開啟 `programs/light_snapshot/light_snapshot.ino`，另存到自己的作品資料夾。重新填入 `PIN_LIGHT`、`PIN_SDA`、`PIN_SCL`、`OLED_ADDRESS`、`OLED_CONTROLLER`；數值沿用 C 已確認的設定，但不會自動帶過來。

另外設定 `PIN_BUTTON`；圖示使用 `GPIO5`。核對完接法才將 `PROFILE_CONFIRMED` 改 `true`。拔 USB、拆掉外接線，只接裸板 USB 上傳。成功後拔 USB，按下列接回：

保留 / 新增	開發板 / 模組接腳與麵包板孔位
保留 C 的兩個模組	光敏 S 腳接開發板 <code>GPIO4</code> ； <code>OLED SDA</code> 、 <code>SCK</code> 腳分別接已確認的 <code>GPIO8</code> 、 <code>GPIO9</code>
GND 分接	開發板 GND 腳 → a3 孔， <code>OLED GND</code> 腳 → b3 孔，光敏 - 腳 → c3 孔
3V3 分接	板 3V3 → a6， <code>OLED VDD</code> → b6，光敏電源 → c6
新增按鈕	e27、f27、e29、f29 跨中央槽
按鈕訊號、接地	<code>GPIO5</code> → a27；a29 → e3

開發板斷電時，用電表通斷檔，兩支筆尖分別碰麵包板 b27、b29 孔的延長導線金屬端；放開按鈕不通，按下才通。關閉電表、移開表筆，核對各列接線後才接 USB 線。Serial Monitor 通訊速率選 115200 baud。

OLED 應顯示 RAW 的目前值、LAST ---、SAVED 0。LAST 是最近保存值，SAVED 是保存次數；剛啟動尚未按過，所以不能顯示一筆虛構的保存值。

目前值繼續變，保存值留在原處

1. 按鈕先放開半秒。按一下，等待畫面，再放開。
2. 觀察 LAST 等於剛保存的 raw、SAVED=1，Monitor 新增一筆 event=saved sample=1 raw=...。
3. 不按按鈕，遮住感測器：RAW 隨光線變，LAST 與 SAVED 保持。
4. 再按一下，LAST 更新、SAVED=2，Monitor 新增第二筆。
5. 持續按住三秒並改變光線：RAW 仍更新，不新增保存紀錄。
6. 放開再按，才保存第三筆；放開後按開發板的 RST 按鈕，LAST 回 ---、SAVED 回 0。

格式示例，非實測：

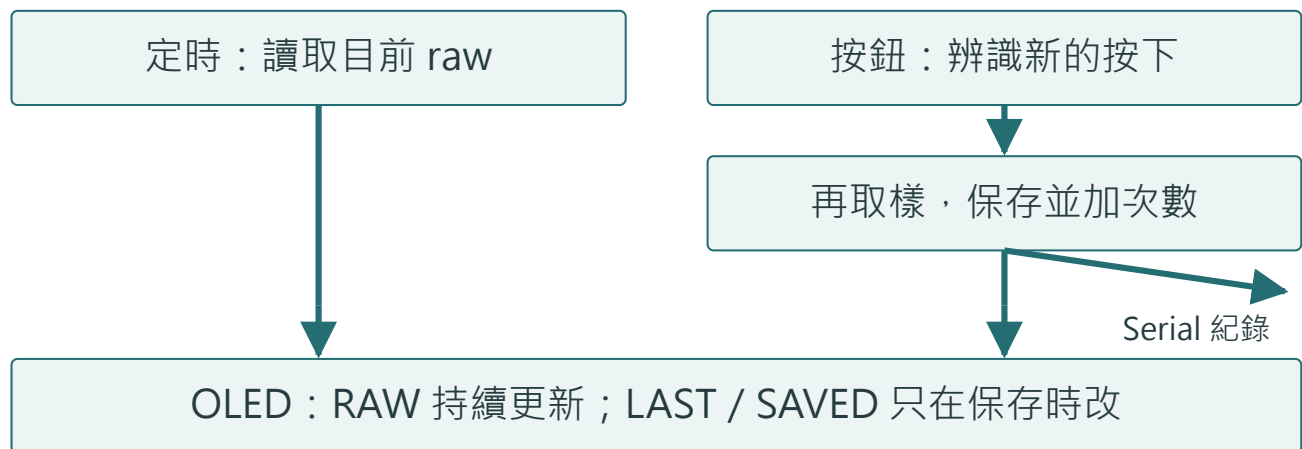
```
event=saved sample=1 raw=480  
event=saved sample=2 raw=980  
event=saved sample=3 raw=510
```

每一筆的 sample 應等於當時畫面的 SAVED，raw 應等於 LAST。RAW 是即時資料，稍後與 LAST 不同很正常。

另試按鈕按住開機：不能自動保存。先放開，再按，才新增第一筆。RST 不會刪掉電腦視窗裡已印出的舊文字，核對新開機的 ready 訊息之後再算筆數。

RAW 來自最近的持續量測，LAST 來自最近一次有效按下時的量測；放開再按才更新 LAST。操作與紀錄一致性以作品展示，不把這裡的 480 / 980 / 510 當成實測。

即時值與保存值，用不同變數記住



箭頭是資訊處理順序，不是接線或電流。

D 的程式說明：raw 放現在的讀值，savedRaw 放最近保存值，savedCount 放保存次數。每到 200 ms 更新 RAW；辨識到新按下時，再讀取當下資料，更新 savedRaw 並加一次 savedCount，立即更新畫面與紀錄。回答時指出自己儲存兩個值的變數及各自更新條件。

若只保存上一張螢幕上的 raw，可能存到按下前的舊資料。本例在按下時重新 analogRead。按壓辨識仍沿用「穩定 40 ms，放開後才接受下一次」，所以按住不放不會反覆保存。

紙上只改一處	結果
每次刷新畫面都改 savedRaw	LAST 追著 RAW 跑，不再代表最近保存值
每次讀到 LOW 都保存	長按會新增許多筆，而不是一筆
只接 OLED 電源，不接 SDA / SCK	模組有電，但收不到程式送的畫面

電流仍由 3V3 分別供應光敏與 OLED，另有晶片上拉電阻 → 按鈕 → GND 的回路。按鈕不替 OLED 供電，也不直接把數字傳給 OLED。

電壓、電流與電阻各用什麼單位？

本週用電表量電壓 (V) 和電阻 (Ω) ；電流 (A) 只在紙上計算。

問什麼	名稱與單位	例子
兩點電位差多少	電壓 V，伏特	3.3 V
電荷流動多少	電流 A，安培	0.01 A = 10 mA
對電流阻礙多少	電阻 Ω ，歐姆	330 Ω ; 1 k Ω = 1000 Ω

單一電阻的紙上例子：330 Ω 兩端相差 3.3 V，則 $I = 3.3 \div 330 = 0.01$ A，不是 0.1 A。若 $R = 1$ Ω ，理想計算才是 3.3 A；沒外接電阻不能自行當成 1 Ω 。

A830L 常見 Ω 量程判讀示例

檔位	示意畫面	解讀
200 Ω	61.0	約 61.0 Ω
20k Ω	1.00	約 1.00 k Ω = 1000 Ω
200k Ω	10.0	約 10.0 k Ω = 10000 Ω
任一 Ω 檔	最左側 1，或其他表的 OL	可能超量程或未接通，不是一般的 1 Ω

實際顯示形式依表型而異，紀錄「61」卻不寫檔位，別人就無法確定單位。電阻檔不是把電壓讀值換成另一個單位。

有聲音就短路嗎？同一五孔組相通，本來就會叫；供電與 GND 原本不該直接相通，卻有低阻路徑，才是需要停止供電排查的情況。

編譯、上傳與感測各確認什麼？

編譯把程式轉成板子能執行的形式；上傳把它寫入板子；感測再用實物驗證。

看到什麼	代表什麼	還沒證明什麼
Verify 沒錯誤	電腦能編譯程式	實際接線、電壓與模組功能
Upload 完成	程式已寫入板子	光線分類必定正確
Monitor 與量測符合測試條件	這輪觀察有對應證據	所有器材、環境與時間都可靠

Flash 存程式，RAM 暫存執行資料

```
Sketch uses 286973 bytes (21%) ... Maximum is 1310720
Global variables use 21936 bytes (6%) ... Maximum is 327680
```

這是編譯摘要格式示例，不是本次程式的編譯結果。

Flash 存程式，斷電仍保留。第一行是在看目前分割設定給程式的空間，不是整顆 16 MB 都拿來當分母。

RAM 是執行時暫存。第二行是編譯時計入的靜態配置，不是之後執行時永遠只用 6%，也不是板上 8 MB PSRAM 的用量。

PSRAM 是額外的 RAM。不是 Flash，也不能由上面兩行就判斷它用了多少。21% 和 6% 分母不同，不能加成整片板用了 27%。

保存量測與程式紀錄

照片、數值、程式與解釋放在同一份紀錄。

保存什麼	對應本冊段落
板型、GPIO、IDE 版本、組別、電表檔位	器材及操作設定
電源 / GPIO 接線照、3V3 一筆、LOW / HIGH 各三筆、Reset 文字	電源及 GPIO 量測
光敏模組 絲印、三組斷電阻值、接線照、S 電壓三筆	光敏模組辨認及電壓
同輪室內 / 遮光各十筆 raw、條件與電壓	原始資料及範圍
四個範圍值、分界方向、各五筆新驗證	分類規則及限制
1 k Ω / 10 k Ω 阻值實測、交換位置前後的電壓與接線照	固定電阻分壓
實際使用的程式、討論及練習答案、尚未完成項目	三支量測 / 分類程式，加上按鈕單筆取樣

整理順序：條件與接法 → 原始資料 → 比較 → 原因與限制。已有資料直接引用，不為換標題再拍照或重收。未完成寫「未量測 / 未確認」，不填理論值。

最後一起斷電收納

1. 移開表筆、關閉 Monitor、拔 USB 與其他電源，確認開發板 PWR 電源燈熄滅。
2. 電表旋鈕轉到 OFF。若停在分壓範例，先拆 b6 → a20，再拆電阻；若已做按鈕取樣，移除光敏模組與按鈕線。若已接 OLED，也移除它的供電與訊號線。
3. 從板子拆下 3V3、GND 線。電阻按阻值收好，模組和線材分開放。

下週再把感測結果接上其他輸出與狀態判斷；本週不把一次分類成功當成可長期自動控制的系統。

操作依據，和還沒做的測試

ADC / GPIO 文件重新核對：2026-09-22；其他來源沿用既有查核。

- [Espressif Arduino-ESP32 ADC](#)：原始值、解析度、衰減與量測範圍。
- [Espressif GPIO](#)：輸入、輸出及 digitalWrite。
- [Espressif ESP32-S3 datasheet](#)：GPIO 與晶片電氣規格。
- [Fluke 電阻量測](#)：斷電與隔離被測物。
- [Fluke 通斷測試](#)：低阻路徑與蜂鳴判讀。
- [Fluke 萬用電表操作](#)：電壓 / 電流插孔不可混用；本課插孔位置另依 A830L 實物照片核對。
- [Analog Devices : Voltage Divider](#)：分壓電路。
- [Arduino Memory Guide](#)：儲存與執行時記憶體。

板卡、光敏模組、電表與電阻照片來自本課既有實物；板型與焊盤狀態依課堂硬體紀錄，不把賣場名稱當作實物規格。示意圖是依接線關係重新繪製，並非實物比例圖。

本稿整理操作順序與版面；**沒有新增上電、接線、電表讀值或感測器實機測試**。示例數字已標明用途，應填入自己觀察的地方仍保留空白。

教師課前須確認使用板型、GPIO5 輸出與 GPIO4 ADC 的適用接法。既有 BOARD-T01 的觀察不代表全班每片板子都已通過；本稿不解除公開程式中 -1 的保護設定。

四支基本程式全文列在各自操作段落。實際上傳開完整 .ino 檔，不只複製中間一頁。Q1 ~ Q5 的參考答案見本冊開頭；後續接著提供 A ~ D 的完整實作。

對照看到的訊息，再查對應位置

看見的情況	先核對什麼
編譯說找不到 U8g2lib.h	程式庫管理員是否已安裝 U8g2
event=blocked reason=check_configuration	該支程式的確認值、腳位、位址；不是另一支的設定
overlapping_ranges	本組同位置兩段資料是否重疊，或最小 / 最大填錯
oled_no_ack	先拔開發板 USB 線，再查 OLED 電源 / 接地 / SDA / SCK 導線與程式中的通訊位址
有 ACK、畫面空白或偏移	控制器與 128×64 解析度是否適用，不提高電壓
遮住後一直加次數	是否讀到雜訊跨過分界，或錯用每次 LOW 都加一的程式
按鈕完全不動作	斷電查按鈕跨組、GPIO5 與 GND，不接 3V3 直通 GND

OLED 程式偵測不到裝置回應時停止更新並留下訊息；修好接線後重新啟動。舊畫面可能仍在，不能只看螢幕有字就認為還在更新。

參考程式每 200 ms 更新 OLED，傳送畫面本身也需要時間。一般半秒按壓是本題測試條件；不保證能捕捉比畫面傳送還短的所有脈衝。

這些程式的設定、主機測試及編譯結果記在維護紀錄。接線圖與畫面數字是教學示例，不代表本次已完成實體 OLED、按鈕或全班器材測試。

對照官方說明與實物

- [Espressif Arduino-ESP32 ADC](#)：analogRead、解析度與量測範圍。
- [Espressif Arduino-ESP32 GPIO](#)：數位輸入與上拉設定。
- [Espressif Arduino-ESP32 I2C](#)：Wire 通訊與裝置回應。
- [U8g2 API](#)：位址格式、字型、畫圖與 sendBuffer。
- [Arduino millis](#)：開機經過時間與時間差的使用。

板卡、模組與目前腳位以已確認的實物紀錄為準；商品標題、I2C 回應或程式成功編譯，都不能取代電氣條件與實際畫面的核對。